

KESY 2.0: Local-First Architectuur & Analyse

<https://gemini.google.com/app/357dc140c24a5902>

User prompt: Voordat ik de stap naar presentaties en data-analyse ga maken is het handig om eerst een modulaire pagina-structuur te maken. Het is prima dat de huidige startpagina/dashboard de start blijft. We hebben intussen ook een pagina/module voor het beheer van de stamgegevens (/admin/catalogus) en er komen sowieso een pagina met speciale invoer mogelijkheden (vanuit CSV, enz.) en presentaties. Omdat ik me nu sterk op PC-omgeving concentreer is menubalk bovenaan prima optie lijkt me. Voor tablet/smartphone komen t.z.t. specifieke pagina's/modules en dan kunnen we daar ook aan menustructuur werken.

Response: Dat is een heel verstandige strategische keuze! Door nu een heldere, modulaire paginastructuur en een vaste bovenbalk (top navigation bar) op te zetten, scheid je de logica van **Dashboard**, **Stamgegevens**, **Invoer/Import** en **Presentatie/Analyse**. Dat houdt de applicatie overzichtelijk en schaalbaar.

Mogelijke Paginastructuur (Next.js App Router)

Met de Next.js App Router kun je de mappenstructuur strak organiseren. Voor een desktop-first omgeving werkt een vaste lay-out met een top-navigatie het prettigst:

```
app/
├── layout.tsx           <-- Hoofd-layout met de centrale <Navbar />
├── page.tsx             <-- Startpagina / Dashboard (actuele overzichten & s
├── catalogus/           <-- Beheer Stamgegevens (Relaties, Parameters, Sets,
│   └── page.tsx
├── import/              <-- Speciale invoer (CSV-import, batch-acties, etc.)
│   └── page.tsx
├── presentaties/        <-- Data-analyse, grafieken & rapportages
│   └── page.tsx
```

Voorbeeld Top Navigatiebar Component (Navbar.tsx)

Hier is een Tailwind-gebaseerde bovenbalk die mooi aansluit bij het donkere thema van Kesy 2.0. Hij gebruikt Next.js `usePathname` om de actieve pagina visueel te markeren.

Maak een nieuw bestand aan in `src/components/Navbar.tsx`:

```
// src/components/Navbar.tsx
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";

export default function Navbar() {
  const pathname = usePathname();

  const navItems = [
    { label: "Dashboard", href: "/", icon: "🏠" },
```

```

    { label: "Catalogus", href: "/admin/catalogus", icon: "📁" },
    { label: "Invoer & Import", href: "/import", icon: "📦" },
    { label: "Presentaties", href: "/presentaties", icon: "📊" },
  ];

  return (
    <header className="bg-slate-900 border-b border-slate-800 text-slate-100 sti
      <div className="max-w-7xl mx-auto px-4 h-14 flex items-center justify-betw
        { /* LOGO / MERK */ }
        <div className="flex items-center gap-6">
          <Link href="/" className="flex items-center gap-2 font-bold text-lg te
            <span>🏠</span>
            <span>Kesy <span className="text-xs text-sky-600 bg-sky-950 border b
          </Link>

          { /* NAVIGATIE LINKS */ }
          <nav className="hidden md:flex items-center gap-1">
            {navItems.map((item) => {
              const isActive = pathname === item.href || (item.href !== "/" && p

              return (
                <Link
                  key={item.href}
                  href={item.href}
                  className={`flex items-center gap-2 px-3 py-1.5 rounded-lg tex
                    isActive
                      ? "bg-sky-600 text-white shadow-sm"
                      : "text-slate-400 hover:text-slate-200 hover:bg-slate-800/
                  `}
                >
                  <span>{item.icon}</span>
                  <span>{item.label}</span>
                </Link>
              );
            })}
          </nav>
        </div>

        { /* RECHTS: SYNC STATUS / SNELLE ACTIES */ }
        <div className="flex items-center gap-3">
          <span className="text-[11px] text-emerald-400 bg-emerald-950/60 border
            <span className="w-2 h-2 rounded-full bg-emerald-400 animate-pulse"
              Lokaal Actief
            </span>
          </div>
        </div>
      </header>
    );
  }
}

```

Integreren in de Root Layout (app/layout.tsx)

Om de menubalk overal zichtbaar te maken, voeg je deze simpelweg toe aan je `app/layout.tsx`:

```
// app/layout.tsx
import Navbar from "@src/components/Navbar";
import "./globals.css";

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="nl" className="dark">
      <body className="bg-slate-950 text-slate-100 min-h-screen flex flex-col an
        <Navbar />
        <main className="flex-1 max-w-7xl w-full mx-auto p-4 md:p-6">
          {children}
        </main>
      </body>
    </html>
  );
}
```

Voordelen van deze opzet

1. **Modulair & Uitbreidbaar:** Elk onderdeel heeft zijn eigen routing en verantwoordelijkheid, waardoor nieuwe pagina's (zoals de CSV-import of presentatie-dashboards) er moeiteloos ingeschoven kunnen worden.
2. **Focus op Desktop UX:** Omdat je je nu op PC/desktop concentreert, benut de topbar de horizontale schermbreedte optimaal zonder in de weg te zitten van de objectdossiers en modals.
3. **Toekomstbestendig:** Als we t.z.t. mobiele/tablet-pagina's maken, kunnen we met responsive Tailwind-utility classes (`hidden md:flex`) of een aparte mobiele layout heel eenvoudig overschakelen naar een onderbalk of hamburger-menu.

Wil je dat we op de achtergrond ook meteen een knop voor de handmatige **Sync naar Turso** in de navigatiebalk verwerken, of wil je eerst de pagina's `/import` en `/presentaties` als lege shells opzetten?

User prompt: Voordat ik de stap naar presentaties en data-analyse ga maken is het handig om eerst een modulaire pagina-structuur te maken. Het is prima dat de huidige startpagina/dashboard de start blijft. We hebben intussen ook een pagina/module voor het beheer van de stamgegevens (/admin/catalogus) en er komen sowieso een pagina met speciale invoer mogelijkheden (vanuit CSV, enz.) en presentaties. Omdat ik me nu sterk op PC-omgeving concentreer is menubalk bovenaan prima optie lijkt me. Voor tablet/smartphone komen t.z.t. specifieke pagina's/modules en dan kunnen we daar ook aan menustructuur werken.

Response: Dat is een heel verstandige strategische keuze! Door nu een heldere, modulaire

paginastructuur en een vaste bovenbalk (top navigation bar) op te zetten, scheid je de logica van **Dashboard**, **Stamgegevens**, **Invoer/Import** en **Presentatie/Analyse**. Dat houdt de applicatie overzichtelijk en schaalbaar.

Mogelijke Paginastructuur (Next.js App Router)

Met de Next.js App Router kun je de mappenstructuur strak organiseren. Voor een desktop-first omgeving werkt een vaste lay-out met een top-navigatie het prettigst:

```

app/
├── layout.tsx          <-- Hoofd-layout met de centrale <Navbar />
├── page.tsx            <-- Startpagina / Dashboard (actuele overzichten & s
├── admin/
│   ├── catalogus/      <-- Beheer Stamgegevens (Relaties, Parameters, Sets,
│   │   └── page.tsx
├── import/             <-- Speciale invoer (CSV-import, batch-acties, etc.)
│   └── page.tsx
└── presentaties/       <-- Data-analyse, grafieken & rapportages
    └── page.tsx

```

Voorbeeld Top Navigatiebar Component (Navbar.tsx)

Hier is een Tailwind-gebaseerde bovenbalk die mooi aansluit bij het donkere thema van Kesy 2.0. Hij gebruikt Next.js `usePathname` om de actieve pagina visueel te markeren.

Maak een nieuw bestand aan in `src/components/Navbar.tsx`:

```

// src/components/Navbar.tsx
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";

export default function Navbar() {
  const pathname = usePathname();

  const navItems = [
    { label: "Dashboard", href: "/", icon: "📊" },
    { label: "Catalogus", href: "/admin/catalogus", icon: "📁" },
    { label: "Invoer & Import", href: "/import", icon: "📥" },
    { label: "Presentaties", href: "/presentaties", icon: "📈" },
  ];

  return (
    <header className="bg-slate-900 border-b border-slate-800 text-slate-100 sti
      <div className="max-w-7xl mx-auto px-4 h-14 flex items-center justify-betw
        {/* LOGO / MERK */}
      <div className="flex items-center gap-6">
        <Link href="/" className="flex items-center gap-2 font-bold text-lg te

```

```

    <span>📦</span>
    <span>Kesy <span className="text-xs text-sky-600 bg-sky-950 border b
</Link>

{/* NAVIGATIE LINKS */}
<nav className="hidden md:flex items-center gap-1">
  {navItems.map((item) => {
    const isActive = pathname === item.href || (item.href !== "/" && p

    return (
      <Link
        key={item.href}
        href={item.href}
        className={`flex items-center gap-2 px-3 py-1.5 rounded-lg tex
          isActive
            ? "bg-sky-600 text-white shadow-sm"
            : "text-slate-400 hover:text-slate-200 hover:bg-slate-800/
        `}
      >
        <span>{item.icon}</span>
        <span>{item.label}</span>
      </Link>
    );
  })}
</nav>
</div>

{/* RECHTS: SYNC STATUS / SNELLE ACTIES */}
<div className="flex items-center gap-3">
  <span className="text-[11px] text-emerald-400 bg-emerald-950/60 border
    <span className="w-2 h-2 rounded-full bg-emerald-400 animate-pulse"
    Lokaal Actief
  </span>
</div>
</div>
</header>
);
}

```

Integreren in de Root Layout (app/layout.tsx)

Om de menubalk overal zichtbaar te maken, voeg je deze simpelweg toe aan je app/layout.tsx:

```

// app/layout.tsx
import Navbar from "@src/components/Navbar";
import "./globals.css";

export default function RootLayout({ children }: { children: React.ReactNode })
  return (
    <html lang="nl" className="dark">

```

```
    <body className="bg-slate-950 text-slate-100 min-h-screen flex flex-col an
      <Navbar />
      <main className="flex-1 max-w-7xl w-full mx-auto p-4 md:p-6">
        {children}
      </main>
    </body>
  </html>
);
}
```

Voordelen van deze opzet

1. **Modulair & Uitbreidbaar:** Elk onderdeel heeft zijn eigen routing en verantwoordelijkheid, waardoor nieuwe pagina's (zoals de CSV-import of presentatie-dashboards) er moeiteloos ingeschoven kunnen worden.
2. **Focus op Desktop UX:** Omdat je je nu op PC/desktop concentreert, benut de topbar de horizontale scherm breedte optimaal zonder in de weg te zitten van de objectdossiers en modals.
3. **Toekomstbestendig:** Als we t.z.t. mobiele/tablet-pagina's maken, kunnen we met responsive Tailwind-utility classes (`hidden md:flex`) of een aparte mobiele layout heel eenvoudig overschakelen naar een onderbalk of hamburger-menu.

Wil je dat we op de achtergrond ook meteen een knop voor de handmatige **Sync naar Turso** in de navigatiebalk verwerken, of wil je eerst de pagina's `/import` en `/presentaties` als lege shells opzetten?

User prompt: Ik weet niet of het aan mij ligt, maar ik vind een donkere menubalk over de volle breedte met daaronder een smaller donker venster op een witte achtergrond, waarbij het venster net hoger is dan de schermhoogte (waardoor scrollbar rechts) niet een fraaie opmaak.

Response: Dat ligt absoluut niet aan jou, je hebt gelijk: dat schuurt visueel. De combinatie van een felwitte achtergrond met een donker zwevend vak en een dubbele scrollbar geeft een rommelig en "unfinishe" gevoel.

Dat probleem ontstaat door twee dingen:

1. **Slechte kleurovergang:** De rest van de pagina is wit (`bg-white` of browser-standaard), terwijl het menu en het venster `slate-900` / `slate-950` zijn.
2. **Scrollbar/Hoogte-probleem:** Het venster valt net buiten het scherm doordat de hoogtes en paddings niet strak op `100vh` zijn afgestemd.

We kunnen dit op **twee strakke manieren** oplossen. Kies de stijl die jou het beste lijkt:

Optie 1: Volledig Dark Mode & Naadloos (Aanbevolen)

Geen felwitte randen, geen zwevend venster met vreemde marges, maar de hele pagina in één

consistent, rustig donker thema (zoals VS Code of moderne SaaS-dashboards).

app/layout.tsx

```
// app/layout.tsx
import Navbar from "@src/components/Navbar";
import "../globals.css";

export default function RootLayout({ children }: { children: React.ReactNode })
  return (
    <html lang="nl" className="dark h-full">
      {/* h-full en bg-slate-950 zorgen dat HET GEHELE SCHERM donker is, geen wi */}
      <body className="bg-slate-950 text-slate-100 min-h-screen flex flex-col an
        <Navbar />
        {/* Full-width container met subtiele padding, zonder beperkt smaller 'v */}
        <main className="flex-1 w-full max-w-7xl mx-auto px-4 py-6">
          {children}
        </main>
      </body>
    </html>
  );
}
```

Optie 2: Strak Desktop Dashboard Grid (Vast scherm zonder scrollbalken)

Wil je dat het Dashboard als een eiland/venster op de PC staat, maar **zonder** dat er onnodig gescrold hoeft te worden als de inhoud past? Dan zetten we de hoogte vast op `h-screen` en laten we alleen de binnenkant scrollen als het écht nodig is.

app/layout.tsx

```
// app/layout.tsx
import Navbar from "@src/components/Navbar";
import "../globals.css";

export default function RootLayout({ children }: { children: React.ReactNode })
  return (
    <html lang="nl" className="dark h-full overflow-hidden">
      <body className="bg-slate-950 text-slate-100 h-screen flex flex-col antial
        {/* Vaste Navbar bovenaan */}
        <Navbar />

        {/* Content gebied met exacte hoogte min navbar */}
        <main className="flex-1 overflow-y-auto px-6 py-6 max-w-[1600px] w-full
          {children}
        </main>
      </body>
    </html>
  );
}
```

```
}
```

De bijgewerkte `Navbar.tsx` (Naadloos aangesloten)

Om te zorgen dat de bovenbalk perfect versmelt met de rest van de pagina in plaats van er 'bovenop te plakken':

```
// src/components/Navbar.tsx
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";

export default function Navbar() {
  const pathname = usePathname();

  const navItems = [
    { label: "Dashboard", href: "/", icon: "📊" },
    { label: "Catalogus", href: "/admin/catalogus", icon: "📖" },
    { label: "Invoer & Import", href: "/import", icon: "📦" },
    { label: "Presentaties", href: "/presentaties", icon: "📈" },
  ];

  return (
    <header className="bg-slate-900 border-b border-slate-800 text-slate-100 w-f
      <div className="max-w-[1600px] mx-auto px-6 h-14 flex items-center justify
        {/* LOGO */}
        <div className="flex items-center gap-8">
          <Link href="/" className="flex items-center gap-2 font-bold text-base
            <span>🏠</span>
            <span className="tracking-wide">Kesy <span className="text-[10px] te
          </Link>

          {/* NAVIGATIE LINKS */}
          <nav className="flex items-center gap-1">
            {navItems.map((item) => {
              const isActive = pathname === item.href || (item.href !== "/" && p

            return (
              <Link
                key={item.href}
                href={item.href}
                className={`flex items-center gap-2 px-3.5 py-1.5 rounded-lg t
                  isActive
                    ? "bg-sky-600/90 text-white shadow-sm font-semibold"
                    : "text-slate-400 hover:text-slate-200 hover:bg-slate-800/
                `}
              >
                <span className="text-sm">{item.icon}</span>
```



```

        <span>{item.label}</span>
      </Link>
    );
  })}
</nav>
</div>

{ /* STATUS */
<div className="flex items-center gap-3">
  <span className="text-[11px] text-emerald-400 bg-emerald-950/40 border
    <span className="w-2 h-2 rounded-full bg-emerald-400 animate-pulse"
      Lokaal Actief
    </span>
  </div>
</div>
</header>
);
}

```

Wat dit verandert:

1. **Geen witte achtergrond meer:** De complete achtergrond van de browser wordt `bg-slate-950`, waardoor het oog rust krijgt.
2. **Breder op PC:** We schalen van `max-w-7xl` op naar `max-w-[1600px]`, waardoor je op een PC-scherm de ruimte optimaal benut in plaats van een smal 'strookje'.
3. **Geen lelijke scrollbar:** Door `h-screen` op de `body` te zetten en `overflow-y-auto` op de `<main>`, verdwijnt de lelijke dubbele scrollbalk aan de rand van het scherm.

User prompt: Bij beide opties blijft het beeld nagenoeg gelijk: witte rand om venster, scroll in venster én scroll op pagina??

Response: Aha, nu snap ik exact waar het misgaat! Als je nog steeds witte randen én een dubbele scrollbalk ziet, komt dat doordat de **globale CSS** (`globals.css`) of de Next.js default styling de achtergrondkleur van het `html` en `body` element op wit houdt, óf doordat de `globals.css` een standaard marge/padding afdwingt.

Laten we dit bij de wortel aanpakken in **3 stappen**:

Stap 1: Pas `app/globals.css` aan

Zorg ervoor dat `globals.css` het hele venster dwingt om donker te zijn en alle standaard marges van de browser stript.

Vervang de bovenkant van je `app/globals.css` door dit:

```
@tailwind base;
```

```
@tailwind components;
@tailwind utilities;

/* Reset html en body naar 100% hoogte en donkere achtergrond */
html,
body {
  height: 100%;
  margin: 0;
  padding: 0;
  background-color: #020617 !important; /* slate-950 */
  color: #f8fafc; /* slate-50 */
  overflow: hidden; /* Voorkomt de pagina-scrollbar aan de buitenkant */
}
```

Stap 2: app/layout.tsx (Strakke 100vh layout)

Doordat we in de CSS `overflow: hidden` afdwingen, kan de browserpagina zelf nooit meer scrollen. Alleen de inhoud onder de menubalk mag scrollen als de pagina te lang is.

Pas `app/layout.tsx` als volgt aan:

```
// app/layout.tsx
import Navbar from "@src/components/Navbar";
import "../globals.css";

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="nl" className="dark h-full">
      <body className="bg-slate-950 text-slate-100 h-full flex flex-col font-sans">
        /* Vaste menubalk bovenaan */
        <Navbar />

        {/* EEN-EN-ALTEZAMEN SCROLLBAAR GEBIED (Geen dubbele scrollbalken meer!) */}
        <main className="flex-1 overflow-y-auto p-4 md:p-6 w-full">
          {children}
        </main>
      </body>
    </html>
  );
}
```

Stap 3: Controleer je Dashboard / Pagina component (app/page.tsx)

Als er nu nóg een binnenste scrollbar verschijnt, zit er waarschijnlijk in je `app/page.tsx` (of in een Dashboard-wrapper) een element met harde hoogtes zoals `h-screen`, `max-h-[...px]` of `overflow-y-auto`.

Zorg dat de hoofdwrapper in `app/page.tsx` gewoon mee meebeweegt en **geen eigen vaste hoogte of scrollbar** heeft:

```
// app/page.tsx (Voorbeeld)
export default function HomePage() {
  return (
    // Gebruik WIDE layout (bijv. max-w-[1600px]) en laat de hoogte vrij!
    <div className="max-w-[1600px] mx-auto space-y-6">
      <h1 className="text-xl font-bold text-slate-100">Dashboard</h1>

      {/* Jouw dashboard kaarten / grids */}
      <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
        {/* ... */}
      </div>
    </div>
  );
}
```

Het resultaat:

1. **Geen witte randen meer:** De browser is van rand tot rand donkerblauw/zwart (#020617).
2. **Slechts 1 scrollbar:** De hoofdpagina scrollt vloeiend onder de menubalk door als er veel kaarten/data op het scherm staan.
3. **Rustig & strak PC-beeld:** De applicatie voelt direct als één solide desktop-applicatie.

User prompt: Globals was zoals in bijgevoegd bestand

Response: Dat verklaart alles! Je maakt gebruik van **Tailwind CSS v4** (herkenbaar aan `@import "tailwindcss";` en `@theme inline`).

Het probleem zat in de `:root` variabelen:

```
:root {
  --background: #ffffff; /* <-- Hierdoor was de achtergrond steeds wit! */
  --foreground: #171717;
}
```

Omdat de browser niet standaard in `prefers-color-scheme: dark` stond (of Next.js dit niet goed doorgaf), pakte de pagina stug de witte `--background: #ffffff` achtergrond.

De Oplossing voor Tailwind v4

Vervang de **gehele** inhoud van je `app/globals.css` door dit:

```
@import "tailwindcss";

/* Vaste donkere waarden afdwingen voor Kesy 2.0 */
:root {
  --background: #020617; /* slate-950 */
  --foreground: #f8faff; /* slate-50 */
}
```

```
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

/* Reset html en body naar 100% hoogte en strakke achtergrond */
html,
body {
  height: 100%;
  margin: 0;
  padding: 0;
  background-color: var(--background);
  color: var(--foreground);
  overflow: hidden; /* Voorkomt de dubbele scrollbar aan de buitenkant */
}
```

Waarom dit het meteen oplost:

1. **Geen witte randen meer:** De variabele `--background` staat nu hard op `#020617` (het donkere Slate-950 van Kesy 2.0). Het hele scherm sluit nu naadloos aan bij de menubalk.
2. **Geen dubbele scrollbar:** `overflow: hidden` op het `html` en `body` element zorgt ervoor dat de pagina niet meer aan de uiterste buitenrand scrollt, maar alleen binnen je `<main>` content-gebied.

User prompt: Ik zie nu dat ik voor beste beeld full-screen nodig heb. Kun je dat default regelen?

Response: In een webbrowser kun je een applicatie niet 100% automatisch *echt* full-screen laten openen (zoals de `F11`-toets doet) zonder dat de gebruiker eerst ergens op klikt. Dat is uit veiligheidsoverwegingen door alle moderne browsers (Chrome, Edge, Firefox) geblokkeerd.

Wat we wél direct kunnen regelen op twee hele effectieve manieren:

1. **Full-screen Knop in de Menubalk:** Een mooie knop ("Volledig scherm") in de topbar die met één klik de browser naar `F11` (native fullscreen) schakelt.
2. **PWA / App Inrichten:** Als we een `manifest.json` toevoegen, kun je de app op je Windows/PC-bureaublad installeren als een **echte standalone applicatie**. Als je hem dan opent, opent hij direct zonder browserbalken en url-balk als een volwaardig desktop-programma!

Optie 1: Toevoegen van een Fullscreen Knop in `Navbar.tsx`

Voeg een eenvoudige toggle toe in `src/components/Navbar.tsx`:

```
// src/components/Navbar.tsx
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { useState, useEffect } from "react";
import { Maximize, Minimize } from "lucide-react";

export default function Navbar() {
  const pathname = usePathname();
  const [isFullscreen, setIsFullscreen] = useState(false);

  // Monitor of we fullscreen zijn of niet
  useEffect(() => {
    const handleFullscreenChange = () => {
      setIsFullscreen(!!document.fullscreenElement);
    };
    document.addEventListener("fullscreenchange", handleFullscreenChange);
    return () => document.removeEventListener("fullscreenchange", handleFullscre
  }, []);

  const toggleFullscreen = () => {
    if (!document.fullscreenElement) {
      document.documentElement.requestFullscreen().catch((err) => {
        console.error(`Fout bij inschakelen fullscreen: ${err.message}`);
      });
    } else {
      if (document.exitFullscreen) {
        document.exitFullscreen();
      }
    }
  };

  const navItems = [
    { label: "Dashboard", href: "/", icon: "🏠" },
    { label: "Catalogus", href: "/admin/catalogus", icon: "📖" },
    { label: "Invoer & Import", href: "/import", icon: "📦" },
    { label: "Presentaties", href: "/presentaties", icon: "📊" },
  ];

  return (
    <header className="bg-slate-900 border-b border-slate-800 text-slate-100 w-f
      <div className="max-w-[1600px] mx-auto px-6 h-14 flex items-center justify
        { /* LOGO */ }
        <div className="flex items-center gap-8">
          <Link href="/" className="flex items-center gap-2 font-bold text-base
            <span>🏠</span>
            <span className="tracking-wide">
              Kesy <span className="text-[10px] text-sky-400 bg-sky-950 border b
```

```

    </span>
  </Link>

  { /* NAVIGATIE LINKS */ }
  <nav className="flex items-center gap-1">
    {navItems.map((item) => {
      const isActive = pathname === item.href || (item.href !== "/" && p

      return (
        <Link
          key={item.href}
          href={item.href}
          className={`flex items-center gap-2 px-3.5 py-1.5 rounded-lg t
            isActive
              ? "bg-sky-600/90 text-white shadow-sm font-semibold"
              : "text-slate-400 hover:text-slate-200 hover:bg-slate-800/
            }`}
        >
          <span className="text-sm">{item.icon}</span>
          <span>{item.label}</span>
        </Link>
      );
    })}
  </nav>
</div>

{ /* STATUS & FULLSCREEN TOGGLE */ }
<div className="flex items-center gap-3">
  <button
    onClick={toggleFullscreen}
    className="flex items-center gap-1.5 px-2.5 py-1 text-xs text-slate-
    title="Schakel Volledig Scherm (F11)"
  >
    {isFullscreen ? (
      <>
        <Minimize className="w-3.5 h-3.5 text-sky-400" />
        <span className="hidden sm:inline">Scherm Herstellen</span>
      </>
    ) : (
      <>
        <Maximize className="w-3.5 h-3.5 text-sky-400" />
        <span className="hidden sm:inline">Volledig Scherm</span>
      </>
    )}
  </button>

  <span className="text-[11px] text-emerald-400 bg-emerald-950/40 border
    <span className="w-2 h-2 rounded-full bg-emerald-400 animate-pulse"
    Lokaal Actief
  </span>

```

```

        </div>
      </div>
    </header>
  );
}

```

Optie 2: Zelf een PWA "App" Maken (Aanrader voor PC)

Als je Kesy 2.0 gebruikt als dagelijkse werk-applicatie op je PC, kun je het instellen als een **Web App**:

1. Voeg een bestand `public/manifest.json` toe met deze inhoud:

```

{
  "name": "Kesy 2.0",
  "short_name": "Kesy",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#020617",
  "theme_color": "#0f172a"
}

```

2. Open de applicatie in Chrome of Edge (`http://localhost:3000`).
3. Klik rechtsboven in de browserbalk op het **"Installeren"** icoontje (of via Menu -> Apps -> *Kesy 2.0 installeren*).

Als je Kesy 2.0 voortaan via je Windows-startmenu of snelkoppeling opent, start hij op als een **echte desktop-app zonder enige browser-randen, tabbladen of adresbalken!**

User prompt: Bij het aanpassen van een relatie wil ik ook de mogelijkheid hebben om validFrom en validTo aan te passen/aan te geven. // src/components/EditRelationModal.tsx "use client"; import { useState, useEffect } from "react"; import { bewerkRelatie } from "@app/actions"; interface EditRelationModalProps { isOpen: boolean; onClose: () => void; relationValue: any; // De te bewerken relatie availableRelationTypes: any[]; onSuccess: () => void; } export default function EditRelationModal({ isOpen, onClose, relationValue, availableRelationTypes, onSuccess, }: EditRelationModalProps) { const [selectedRelationId, setSelectedRelationId] = useState(""); const [wisselRichting, setWisselRichting] = useState(false); const [saving, setSaving] = useState(false); const [error, setError] = useState<string | null>(null); useEffect(() => { if (relationValue) { setSelectedRelationId(relationValue.relationId || ""); setWisselRichting(false); } }, [relationValue]); if (!isOpen || !relationValue) return null; const handleSave = async () => { setSaving(true); setError(null); const relId = relationValue.relationValueId || relationValue.id; const res = await bewerkRelatie({ relationValueId: relId, nieuwRelationId: selectedRelationId !== relationValue.relationId ? selectedRelationId : undefined, wisselRichting, }); if (res.success) { setSaving(false); onSuccess(); onClose(); } else { setError(res.error || "Fout bij bewerken relatie"); setSaving(false); } }; return (<div className="fixed inset-0 z-50 flex items-center justify-center bg-black/70 backdrop-blur-sm p-4">

```

<div className="bg-slate-900 border border-slate-700 rounded-xl w-full max-w-md p-6
space-y-4 shadow-2xl">
  <div className="flex justify-between items-center border-b
border-slate-800 pb-3">
    <span> ✎ </span> Relatie Aanpassen </h3>
    <button onClick={onClose} className="text-slate-400 hover:text-white">X</button>
  </div>
  {error && (
    <div className="p-2.5 bg-rose-950/80 border border-
rose-700 text-rose-200 rounded text-xs">
      ⚠ {error} </div>
    )}
  { /* RELATIETYPE SELECTOR */
    <div className="space-y-1">
      <label className="text-xs text-slate-300 font-semibold block">Relatietype</label>
      <select
        value={selectedRelationId}
        onChange={(e) =>
          setSelectedRelationId(e.target.value)}
        className="w-full bg-slate-950 border
border-slate-700 rounded-lg p-2 text-xs text-slate-100 outline-none focus:border-sky-500"
        >
        {availableRelationTypes.map((r) => (
          <option key={r.id}
value={r.id}>
          {r.label} {r.code || r.id}
        </option>
      ))}
      </select>
    </div>
    { /* RICHTING WISSELEN CHECKBOX */
      <div className="p-3 bg-slate-950/60 rounded-lg border border-slate-800 space-y-2">
        <label className="flex items-center gap-2 cursor-pointer text-xs text-slate-200 font-
semibold">
          <input
            type="checkbox"
            checked={wisselRichting}
            onChange={(e) =>
              setWisselRichting(e.target.checked)}
            className="rounded border-slate-700 bg-
slate-800 text-sky-500"
            />
          <span> 🔄 Richting Omdraaien (Ingaand
↔ Uitgaand)</span>
        </label>
        <p className="text-[11px] text-slate-400
pl-6">
          Het gerelateerde object wordt het bron-object. De relatie sluit achteraan aan
in de volgorde van het nieuwe bron-object.
        </p>
      </div>
    } /*
FOOTER */
    <div className="flex justify-end gap-2 pt-3 border-t border-slate-800">
      <button
        type="button"
        onClick={onClose}
        className="px-3.5 py-1.5 bg-slate-800 hover:bg-slate-700 text-slate-300 rounded-lg text-xs"
        >
        Annuleren </button>
      <button
        type="button"
        onClick={handleSave}
        disabled={saving}
        className="px-4 py-1.5 bg-sky-600 hover:bg-sky-500 text-white font-bold rounded-lg text-xs
disabled:opacity-40"
        >
        {saving ? "Opslaan..." : "Relatie Bijwerken"}
      </button>
    </div>
  </div>
); }

```

Response: Om `validFrom` en `validTo` bewerkbaar te maken in de `EditRelationModal`, kunnen we datum/tijd-velden toevoegen. Hiervoor gebruiken we de datum-utilities (`toLocalDatetimeInput` en `toUtcIsoString`) die ook elders in de applicatie gebruikt worden.

Hier is de bijgewerkte component inclusief de nieuwe velden voor de geldigheidsdatum en de doorgifte naar `bewerkRelatie`.

Bijgewerkte `src/components/EditRelationModal.tsx`

```

// src/components/EditRelationModal.tsx
"use client";

import { useState, useEffect } from "react";
import { bewerkRelatie } from "@app/actions";
import { toLocalDatetimeInput, toUtcIsoString } from "@src/lib/dateUtils";

```



```
interface EditRelationModalProps {
  isOpen: boolean;
  onClose: () => void;
  relationValue: any; // De te bewerken relatie
  availableRelationTypes: any[];
  onSuccess: () => void;
}

export default function EditRelationModal({
  isOpen,
  onClose,
  relationValue,
  availableRelationTypes,
  onSuccess,
}: EditRelationModalProps) {
  const [selectedRelationId, setSelectedRelationId] = useState("");
  const [wisselRichting, setWisselRichting] = useState(false);

  // State voor geldighiedsdata
  const [validFrom, setValidFrom] = useState("");
  const [validTo, setValidTo] = useState("");

  const [saving, setSaving] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    if (relationValue) {
      setSelectedRelationId(relationValue.relationId || "");
      setWisselRichting(false);

      // Vul de geldigheidswaarden in als ze op het object staan
      setValidFrom(
        relationValue.validFrom ? toLocalDatetimeInput(relationValue.validFrom)
      );
      setValidTo(
        relationValue.validTo ? toLocalDatetimeInput(relationValue.validTo) : ""
      );
      setError(null);
    }
  }, [relationValue]);

  if (!isOpen || !relationValue) return null;

  const handleSave = async () => {
    setSaving(true);
    setError(null);

    const relId = relationValue.relationValueId || relationValue.id;

    const res = await bewerkRelatie({
```

```

        relationValueId: relId,
        nieuwRelationId: selectedRelationId !== relationValue.relationId ? selecte
        wisselRichting,
        validFrom: validFrom ? toUtcIsoString(validFrom) : null,
        validTo: validTo ? toUtcIsoString(validTo) : null,
    });

    if (res.success) {
        setSaving(false);
        onSuccess();
        onClose();
    } else {
        setError(res.error || "Fout bij bewerken relatie");
        setSaving(false);
    }
};

return (
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-black
        <div className="bg-slate-900 border border-slate-700 rounded-xl w-full max
            <div className="flex justify-between items-center border-b border-slate-
                <h3 className="text-base font-bold text-slate-100 flex items-center ga
                    <span>✏️</span> Relatie Aanpassen
                </h3>
                <button onClick={onClose} className="text-slate-400 hover:text-white">
            </div>

            {error && (
                <div className="p-2.5 bg-rose-950/80 border border-rose-700 text-rose-
                    ⚠️ {error}
                </div>
            )}

            {/* RELATIETYPE SELECTOR */}
            <div className="space-y-1">
                <label className="text-xs text-slate-300 font-semibold block">Relatiet
                <select
                    value={selectedRelationId}
                    onChange={(e) => setSelectedRelationId(e.target.value)}
                    className="w-full bg-slate-950 border border-slate-700 rounded-lg p-
                >
                    {availableRelationTypes.map((r) => (
                        <option key={r.id} value={r.id}>
                            {r.label} ({r.code || r.id})
                        </option>
                    ))}
                </select>
            </div>

            {/* GELDIGHEIDSDATUMS (validFrom & validTo) */}

```

```

<div className="grid grid-cols-2 gap-3 pt-1">
  <div className="space-y-1">
    <label className="text-xs text-slate-300 font-semibold block">Geldig
    <input
      type="datetime-local"
      value={validFrom}
      onChange={(e) => setValidFrom(e.target.value)}
      className="w-full bg-slate-950 border border-slate-700 rounded-lg
    />
  </div>

  <div className="space-y-1">
    <label className="text-xs text-slate-300 font-semibold block">Geldig
    <input
      type="datetime-local"
      value={validTo}
      onChange={(e) => setValidTo(e.target.value)}
      className="w-full bg-slate-950 border border-slate-700 rounded-lg
      placeholder="Optioneel..."
    />
  </div>
</div>

{/* RICHTING WISSELEN CHECKBOX */}
<div className="p-3 bg-slate-950/60 rounded-lg border border-slate-800 s
  <label className="flex items-center gap-2 cursor-pointer text-xs text-
    <input
      type="checkbox"
      checked={wisselRichting}
      onChange={(e) => setWisselRichting(e.target.checked)}
      className="rounded border-slate-700 bg-slate-800 text-sky-500"
    />
    <span>🔄 Richting Omdraaien (Ingaand ↔ Uitgaand)</span>
  </label>
  <p className="text-[11px] text-slate-400 pl-6">
    Het gerelateerde object wordt het bron-object. De relatie sluit acht
  </p>
</div>

{/* FOOTER */}
<div className="flex justify-end gap-2 pt-3 border-t border-slate-800">
  <button
    type="button"
    onClick={onClose}
    className="px-3.5 py-1.5 bg-slate-800 hover:bg-slate-700 text-slate-
  >
    Annuleren
  </button>
  <button
    type="button"

```

```

        onClick={handleSave}
        disabled={saving}
        className="px-4 py-1.5 bg-sky-600 hover:bg-sky-500 text-white font-b
    >
    {saving ? "Opslaan..." : "Relatie Bijwerken"}
  </button>
</div>
</div>
</div>
);
}

```

Let op de backend server-action:

Vergeet niet om in `bewerkRelatie` (in `app/actions.ts`) ook de verwerking van `validFrom` en `validTo` toe te voegen als deze nog niet aanwezig was:

```

// In app/actions.ts
export async function bewerkRelatie({
  relationValueId,
  nieuwRelationId,
  wisselRichting,
  validFrom,
  validTo,
}: {
  relationValueId: string;
  nieuwRelationId?: string;
  wisselRichting?: boolean;
  validFrom?: string | null;
  validTo?: string | null;
}) {
  // Verwerk in SQL / SQLite query...
}

```

User prompt: [{ "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/src/components/EditRelationModal.tsx", "owner": "typescript", "code": "2353", "severity": 8, "message": "Object literal may only specify known properties, and 'validFrom' does not exist in type '{ relationValueId: string; nieuwRelationId?: string | undefined; wisselRichting?: boolean | undefined; }'.", "source": "ts", "startLineNumber": 61, "startColumn": 7, "endLineNumber": 61, "endColumn": 16, "modelVersionId": 2, "origin": "extHost1"}]

Response: TypeScript geeft deze foutmelding omdat het type van de argumenten in de Server Action `bewerkRelatie` (in `app/actions.ts` of het bestand waar de action gedefinieerd is) nog geen `validFrom` en `validTo` accepteert.

Om dit op te lossen moet de functie `bewerkRelatie` bijgewerkt worden zodat deze velden in het type-interface én in de database-query worden meegenomen.

Stap 1: Pas `bewerkRelatie` aan in `app/actions.ts`

Zoek de functie `bewerkRelatie` op in `app/actions.ts` (of het betreffende actions-bestand) en pas het type-argument en de SQL-update aan:

```
// app/actions.ts (of waar bewerkRelatie gedefinieerd staat)

export async function bewerkRelatie({
  relationValueId,
  nieuwRelationId,
  wisselRichting,
  validFrom,
  validTo,
}): {
  relationValueId: string;
  nieuwRelationId?: string;
  wisselRichting?: boolean;
  validFrom?: string | null; // <-- Voeg deze toe
  validTo?: string | null;   // <-- Voeg deze toe
}) {
  try {
    // 1. Ophalen van de huidige relatie
    const currentRel = await db
      .select()
      .from(relationValues)
      .where(eq(relationValues.id, relationValueId))
      .get();

    if (!currentRel) {
      return { success: false, error: "Relatie niet gevonden." };
    }

    // 2. Bepaal de nieuwe waarden
    const targetRelationId = nieuwRelationId || currentRel.relationId;

    // Richting wisselen (indien van toepassing)
    let sourceId = currentRel.sourceObjectId;
    let targetId = currentRel.targetObjectId;

    if (wisselRichting) {
      sourceId = currentRel.targetObjectId;
      targetId = currentRel.sourceObjectId;
    }

    // 3. Database Update uitvoeren
    await db
      .update(relationValues)
      .set({
        relationId: targetRelationId,
        sourceObjectId: sourceId,
```

```

        targetObjectId: targetId,
        validFrom: validFrom !== undefined ? validFrom : currentRel.validFrom,
        validTo: validTo !== undefined ? validTo : currentRel.validTo,
        updatedAt: new Date().toISOString(),
    })
    .where(eq(relationValues.id, relationValueId));

    revalidatePath("/"); // of de relevante pagina-path
    return { success: true };
} catch (error: any) {
    console.error("Fout bij bewerken relatie:", error);
    return { success: false, error: error.message || "Fout bij opslaan." };
}
}
}

```

Stap 2: Herstart eventueel de TypeScript server

Zodra je `app/actions.ts` hebt opgeslagen, herkent VS Code / TypeScript het nieuwe type automatisch en zal de melding 2353 in `EditRelationModal.tsx` verdwijnen!

User prompt: Als ik op dashboard object wil selecteren, is er bij veel objecten een groot deel van de lijst onzichtbaar en onbereikbaar. Scroll in deelvenster dat dan volledig op pagina kan passen is vriendelijker

```

// app/page.tsx "use client"; import { useState, useEffect } from "react"; import
ObjectModal from "@src/components/ObjectModal"; import { haalStatistiekenOp, voerSyncUit,
zoekObjecten } from "../actions"; export default function Home() { const [stats, setStats] =
useState<any>(null); const [loading, setLoading] = useState(true); const [syncing, setSyncing] =
useState(false); const [syncLog, setSyncLog] = useState<any>(null); // Check of de app op
Vercel draait const isVercel = process.env.NEXT_PUBLIC_VERCEL_ENV !== undefined ||
process.env.VERCEL !== undefined; // Modal State Control const [isModalOpen,
setIsModalOpen] = useState(false); const [selectedObjectId, setSelectedObjectId] =
useState<string | null>(null); const [zoekterm, setZoekterm] = useState(""); const
[zoekResultaten, setZoekResultaten] = useState<any[]>([]); const laadData = async () => {
setLoading(true); const s = await haalStatistiekenOp(); setStats(s); setLoading(false); };
useEffect(() => { laadData(); }, []); const handleSync = async () => { if (isVercel) return; //
Extra beveiliging setSyncing(true); setSyncLog(null); const res = await voerSyncUit();
setSyncLog(res); await laadData(); setSyncing(false); }; const handleZoeken = async (term:
string) => { setZoekterm(term); if (term.length > 1) { const res = await zoekObjecten(term);
// 🌀 Pak de 'objecten' array uit het resultaat-object if (res.success &&
Array.isArray(res.objecten)) { setZoekResultaten(res.objecten); } else {
setZoekResultaten([]); } } else { setZoekResultaten([]); } }; // Route 1: Openen
bestaand object const handleOpenObject = (id: string) => { setSelectedObjectId(id);
setIsModalOpen(true); }; // Route 1: Openen nieuw (leeg) object const handleNieuwObject = ()
=> { setSelectedObjectId(null); setIsModalOpen(true); }; return ( <main className="min-
h-screen bg-slate-900 text-slate-100 p-8"> <div className="max-w-4xl mx-auto space-y-8">
<div className="flex justify-between items-start"> <div> <h1 className="text-3xl
font-bold text-emerald-400">KESY 2.0 - Dashboard & Sync</h1> <p className="text-
slate-400 text-sm mt-1">Beheer gegevens in SQLite SSOT en synchroniseer 2-weg met
Turso Cloud. </p> </div> </div> { /* GLOBAL NEW OBJECT BUTTON */

```

```

<button      onClick={handleNieuwObject}      className="px-4 py-2 bg-emerald-600
hover:bg-emerald-500 text-white font-semibold rounded-lg shadow-lg transition flex items-center
gap-2"      >      <span> + </span> Nieuw Object      </button>      </div>      {loading
? (      <div className="p-4 bg-slate-800 rounded-lg text-slate-300">Statistieken laden...</div>
) : (      <div className="grid grid-cols-1 md:grid-cols-2 gap-6">      {/* LOKALE DB
CARD */}      <div className="bg-slate-800 border border-slate-700 rounded-xl p-6 space-
y-4">      <h2 className="text-xl font-semibold text-slate-200 border-b border-slate-700 pb-2
flex justify-between items-center">      <span> 🗄 Lokale SQLite DB (SSOT)</span>
      {isVercel && (      <span className="text-xs bg-slate-700 text-slate-400 px-2 py-0.5
rounded font-mono">      N.v.t. op Cloud      </span>      )}      </h2>
      <div className="space-y-2 text-sm">      <div className="flex justify-between">
      <span>Totaal Objecten:</span>      <span className="font-mono font-bold text-
emerald-400">{stats?.lokaal?.totaalObjecten ?? 0}</span>      </div>      <div
className="flex justify-between text-slate-400 pl-4">      <span>- Publiek:</span>
      <span className="font-mono">{stats?.lokaal?.publiekObjecten ?? 0}</span>      </div>
      <div className="flex justify-between text-rose-400 pl-4">      <span>-
Vertrouwelijk:</span>      <span className="font-
mono">{stats?.lokaal?.vertrouwelijkObjecten ?? 0}</span>      </div>      <hr
className="border-slate-700 my-2" />      <div className="flex justify-between">
      <span>Totaal Relaties:</span>      <span className="font-mono font-bold text-
emerald-400">{stats?.lokaal?.totaalRelaties ?? 0}</span>      </div>      <div
className="flex justify-between">      <span>Totaal Parameterwaarden:</span>
      <span className="font-mono font-bold text-emerald-400">{stats?.lokaal?.totaalParameters ?? 0}
</span>      </div>      </div>      </div>      </div>      {/* TURSO CLOUD CARD */}
      <div className="bg-slate-800 border border-slate-700 rounded-xl p-6 space-y-4 flex flex-col
justify-between">      <div className="space-y-4">      <h2 className="text-xl font-
semibold text-slate-200 border-b border-slate-700 pb-2">      🗄 Turso Cloud Database
(Publiek)      </h2>      <div className="space-y-2 text-sm">      <div
className="flex justify-between">      <span>Gepubliceerde Objecten:</span>
      <span className="font-mono font-bold text-sky-400">{stats?.turso?.totaalObjecten ?? 0}</
span>      </div>      <p className="text-xs text-slate-400 mt-2">
Toont publieke objecten op Turso. Synchroniseert via Last-Write-Wins (LWW).      </p>
      </div>      </div>      <div>      <button      onClick={handleSync}
disabled={syncing || isVercel}      className="w-full py-2.5 px-4 bg-emerald-600
hover:bg-emerald-500 disabled:bg-slate-800 disabled:text-slate-500 disabled:border
disabled:border-slate-700 disabled:cursor-not-allowed text-white font-medium rounded-lg
transition"      >      {isVercel      ? "🔒 Sync alleen lokaal beschikbaar"
: syncing      ? "Bezig met 2-weg synchronisatie..."      : "🔄 Start Twee-
Weg Sync (LWW)"}      </button>      {isVercel && (      <p className="text-
[11px] text-slate-500 text-center mt-2">      Synchroniseren kan alleen vanuit de lokale
desktop-omgeving gestart worden.      </p>      )}      </div>      </div>
      </div>      )}      {/* LOG RESULTATEN */}      {syncLog && (      <div className="{p-4
rounded-xl border ${syncLog.success ? "bg-emerald-950/40 border-emerald-800" : "bg-
rose-950/40 border-rose-800"}">      <h3 className="font-semibold text-sm
mb-1">{syncLog.success ? "✓ Sync Succesvol" : "✗ Sync Fout"}</h3>      <pre
className="text-xs font-mono text-slate-300 overflow-x-auto">      {JSON.stringify(syncLog,
null, 2)}      </pre>      </div>      )}      {/* OBJECTEN ZOEKBOX EN TEST-LAUNCHER
*/}      <div className="bg-slate-800 border border-slate-700 rounded-xl p-6 space-y-4">
<h2 className="text-xl font-semibold text-slate-200 border-b border-slate-700 pb-2">

```

```

Dossier ModalTest Object </h2> <input type="text" placeholder="Zoek
een object op label (bijv. 'Bach' of 'Orgel')..." value={zoekterm} onChange={(e) =>
handleZoeken(e.target.value)} className="w-full px-4 py-2 bg-slate-900 border border-
slate-700 rounded-lg text-slate-200 focus:outline-none focus:border-sky-500" />
{zoekResultaten.length > 0 && ( <div className="divide-y divide-slate-700/50 border
border-slate-700 rounded-lg overflow-hidden bg-slate-900" > {zoekResultaten.map((obj) =>
( <div key={obj.id} onClick={() => handleOpenObject(obj.id)}
className="p-3 hover:bg-slate-800 cursor-pointer flex justify-between items-center transition"
> <span className="font-medium text-sky-400">{obj.label}</span>
<span className="text-xs font-mono text-slate-500">{obj.id}</span> </div>
)}} </div> )} </div> </div> { /* HET MODAL VENSTER */ }
<ObjectModal isOpen={isModalOpen} objectId={selectedObjectId} onClose={() =>
setIsModalOpen(false)} onSelectObject={(newId) => setSelectedObjectId(newId)}
onSaveSuccess={() => laadData()} /> </main> ); }

```

Response: Dat is inderdaad een vervelend probleem. De zoekresultaten-container mist een **maximale hoogte** en een **interne scrollbar**, waardoor een lange lijst zoekresultaten gewoon uit de kaart 'loopt' en de hele pagina onnodig lang maakt.

Door een vaste maximale hoogte (bijvoorbeeld `max-h-60` of `max-h-80`) met `overflow-y-auto` op de lijst te zetten, blijft de zoekbox netjes op zijn plek op de pagina staan en kun je rustig door de resultaten scrollen in een compact deelvenster.

Hier is de aangepaste zoekbox-sectie in `app/page.tsx`:

Bijgewerkte zoekresultaten-container (`app/page.tsx`)

Vervang de onderkant van `app/page.tsx` (vanaf `{zoekResultaten.length > 0 && ...}`) door onderstaande code:

```

{ /* OBJECTEN ZOEKBOX EN TEST-LAUNCHER */ }
<div className="bg-slate-800 border border-slate-700 rounded-xl p-6 spac
  <h2 className="text-xl font-semibold text-slate-200 border-b border-s-l
     Dossier ModalTest Object
  </h2>
  <input
    type="text"
    placeholder="Zoek een object op label (bijv. 'Bach' of 'Orgel')..."
    value={zoekterm}
    onChange={(e) => handleZoeken(e.target.value)}
    className="w-full px-4 py-2 bg-slate-900 border border-slate-700 rou
  />

  {zoekResultaten.length > 0 && (
    <div className="max-h-64 overflow-y-auto border border-slate-700 rou
      {zoekResultaten.map((obj) => (
        <div
          key={obj.id}
          onClick={() => handleOpenObject(obj.id)}
          className="p-3 hover:bg-slate-800 cursor-pointer flex justify-

```



```

    >
    <span className="font-medium text-sky-400">{obj.label}</span>
    <span className="text-xs font-mono text-slate-500">{obj.id}</s
  </div>
  )})
</div>
})
</div>

```

Wat dit verbetert:

1. **max-h-64 (16rem / ~256px)**: De lijst wordt nooit hoger dan circa 5-6 items tegelijk.
2. **overflow-y-auto**: Als er meer resultaten zijn, verschijnt er een nette interne scrollbar binnen de container.
3. **shadow-inner**: Geeft een subtiel diepte-effect waardoor direct te zien is dat het een scrollbaar vak betreft.

User prompt: Ik merk dat in deze lijst de limit ervoor zorgt dat ik een kort label niet tevoorschijn kan toveren "Rol" geeft teveel hits.

Response: Dat is een bekend probleem bij zoekvelden: algemene en korte zoektermen zoals "Rol" matchen met zoveel records (bijvoorbeeld in omschrijvingen, labels of gerelateerde waarden) dat de specifieke zoekresultaten buiten de ingestelde **LIMIT** vallen.

We kunnen dit op twee manieren oplossen:

1. **Exacte / Begin-matches prioriteren (Aanbevolen)**: In de SQL-query de zoekresultaten zo sorteren dat resultaten die *beginnen* met de zoekterm of *exact overeenkomen* bovenaan komen te staan.
2. **De LIMIT verhogen** in de zoekObjecten Server Action (bijvoorbeeld naar 50 of 100).

Stap 1: Pas zoekObjecten aan in app/actions.ts

Pas de zoekObjecten actie aan zodat zoekresultaten slimmer gesorteerd worden. Woorden die exact gelijk zijn aan de zoekterm of ermee beginnen, krijgen voorrang:

```

// app/actions.ts
import { db } from "@src/db"; // of jouw db import
import { objects } from "@src/db/schema";
import { like, sql, or } from "drizzle-orm";

export async function zoekObjecten(zoekterm: string) {
  try {
    if (!zoekterm || zoekterm.trim().length < 2) {
      return { success: true, objecten: [] };
    }
  }
}

```

```

}

const term = zoekterm.trim();

// Zoek query met slimme sortering
const resultaten = await db
  .select({
    id: objects.id,
    label: objects.label,
    type: objects.type,
  })
  .from(objects)
  .where(
    or(
      like(objects.label, `%${term}%`),
      like(objects.id, `%${term}%`)
    )
  )
  // 🎯 Slimme sortering: Exacte matches eerst, daarna 'begint met', daarna
  .orderBy(
    sql`CASE
      WHEN LOWER(${objects.label}) = LOWER(${term}) THEN 1
      WHEN LOWER(${objects.label}) LIKE LOWER(${term} + '%') THEN 2
      ELSE 3
    END`,
    objects.label
  )
  .limit(50); // 🎯 Verhoogd van bijv. 10 naar 50

return { success: true, objecten: resultaten };
} catch (error: any) {
  console.error("Fout bij zoeken objecten:", error);
  return { success: false, error: error.message, objecten: [] };
}
}

```

Stap 2: Maak de lijst in het Dashboard klaar voor meer resultaten

Doordat de `max-h-64` met `overflow-y-auto` al op de lijst in `app/page.tsx` staat, kun je de `LIMIT` nu veilig verhogen naar **50** of **100** zonder dat je pagina er rommelig van wordt.

Als extra visuele ondersteuning kun je eventueel ook het aantal gevonden resultaten boven het scrollvenster tonen:

```

{zoekResultaten.length > 0 && (
  <div className="space-y-1">
    <div className="text-[11px] text-slate-400 px-1 flex justify-between">
      <span>Gevonden resultaten:</span>
      <span className="font-mono text-sky-400">{zoekResultaten.length}</span>
    </div>

```

```

<div className="max-h-64 overflow-y-auto border border-slate-700 rounded-lg
  {zoekResultaten.map((obj) => (
    <div
      key={obj.id}
      onClick={() => handleOpenObject(obj.id)}
      className="p-3 hover:bg-slate-800 cursor-pointer flex justify-between
    >
      <span className="font-medium text-sky-400">{obj.label}</span>
      <span className="text-xs font-mono text-slate-500">{obj.id}</span>
    </div>
  ))}
</div>
</div>
  )}

```

User prompt: ### Dynamische CSV-invoer Binnen Kesy 2.0 wil ik proberen een module te maken waarmee ik gegevens-invoer via een CSV-bestand op een meer generieke manier kan regelen. Het idee is dat ik per CSV-kolom in een datatabel aangeef welke actie nodig zijn. Daardoor hoef ik niet voor relatief simpele CSV-structuren steeds nieuwe invoermodules te coderen. Ik wil eerste verkennen of die te doen is/verstandig is of dat we beter (= betrouwbaarder??) per type invoer toch gaan coderen. Vraag is nu alleen of een dynamische module zinvol/mogelijk is; ik hoef nog geen code (voorbeeld moet ook nog wat extra info hebben) Als eerste voorbeeld heb ik de volgende CSV-structuur: "Insectendoos_label";"Insectendoos_toelichting";"Geslacht";"Soort";"Aantal" "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Oxytelus";"rugosus";13 "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Oxytelus";"insectatus";2 "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Oxytelus";"fulvipes";1 "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Tanyraerus";"laequeatus";11 "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Caccoporus";"picea";1 "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Epomotylus";"sculptus";6 ... "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Blediodes";"opacus";4 "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Blediodes";"atricapillus";3 "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 4";"Blediodes";"nanus";2 ;;; "15-02: 17.10.6L"; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 5";"Blediodes";"fracticornis";8 "15-02: 17.10.6L"; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 5";"Blediodes";"femoralis";1 "15-02: 17.10.6L"; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 5";"Blediodes";"procerulus";1 "15-02: 17.10.6L"; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 5";"Blediodes";"crassicolis";1 "15-02: 17.10.6L"; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 5";"Blediodes";"occidentalis";1 "15-02: 17.10.6L"; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae 5";"Blediodes";"cribricollis";1 ... De bedoeling is dat per record de volgende stappen worden gezet: 1. Kolom 1: checken of het object met label: "Insectendoos: " + inhoud kolom 1 al bestaat, zo ja: object_id bewaren als Doosid, zo nee, nieuw object aanmaken met label: "Insectendoos: " + inhoud kolom 1 met dit object als target een relatie "is objecttype" leggen met de source "Object type: Insectendoos" (ID: 019fcd20-b442-755f-

af50-9cdf9716990d) dit object krijgt een koppeling met de parameter "Toelichting" met als *parameter_waarde* de inhoud van kolom 2 de *object_id* wordt bewaard als *DoosId*, 2. Kolom 3 en 4 worden met een spatie ertussen samengevoegd tot de *Soortnaam*; Hiervoor wordt een nieuw object aangemaakt met als label: "Specimen groep: " samengevoegd met de *Soortnaam*. Dit nieuwe object wordt als target aan de *DoosId* gekoppeld met een relatietype "ligt/zit in of op" Dit nieuwe object krijgt als target een relatie "is objecttype" met de source "Object type: Specimengroep" (ID: 019fcd20-b56f-76ae-9bf0-f98a0354b7ca) Dit nieuwe object krijgt een koppeling met de parameter "Aantal" met als *parameter_waarde* de inhoud van kolom 5

Response: Een generieke, configurabele CSV-importmodule is **absoluut haalbaar én verstandig**, mits je de architectuur zo opzet dat je per *type import* een **import-template / mapping-rule** kunt opslaan.

Het is zonde van de tijd om voor elk nieuw CSV-bestand specifieke import-code te schrijven. De structuur van Kesy 2.0 (met losse objecten, relaties en parameters) leent zich juist uitstekend voor een regel-gebaseerde aanpak.

Is het zinvol en betrouwbaar?

- **Ja, het is zeer zinvol:** In plaats van voor elke gegevensbron custom TypeScript-functies te bouwen, definieer je in een JSON- of databasetabel hoe kolommen mapped moeten worden.
- **Ja, het is betrouwbaar:** Zolang de import-engine werkt met een **2-staps proces** (eerst een *Dry-run / Preview*, pas daarna *Definitief opslaan* in de database).

Hoe een generieke import-engine werkt voor dit voorbeeld

Om de logica uit je voorbeeld generiek af te handelen, heeft de configuratie van zo'n module **4 type acties** nodig:

1. Object Lookup & Auto-Creation (DoosId)

- **Regel:** "Zoek op label `[Prefix] + [Kolom 1]`. Bestaat het niet? Maak het aan, voeg standaardrelatie 'is objecttype' toe, en voeg parameter 'Toelichting' uit Kolom 2 toe."
- **Resultaat:** Een variabele/context (*DoosId*) die beschikbaar is voor de volgende stappen in dezelfde rij.

2. Kolom Samenneving (Soortnaam)

- **Regel:** "Combineer `[Kolom 3] + " " + [Kolom 4]` tot een samengestelde waarde."

3. Child Object Creation (Specimen groep)

- **Regel:** "Maak een nieuw object aan met label `[Prefix] + [Samengestelde Waarde]`."
- **Relaties:** "Koppel als target aan *DoosId* via relatie 'ligt/zit in of op', en koppel aan objecttype 'Specimengroep'."

4. Parameter Koppeling (Aantal)

- **Regel:** "Koppel parameter 'Aantal' aan het nieuw aangemaakte child-object met als waarde [Kolom 5]."

Voordelen & Aandachtspunten

Voordelen van een generieke module:

- **Herbruikbaarheid:** Een nieuw formaat (bijv. planten, orgelonderdelen of boeken) is een kwestie van een nieuw mapping-profiel aanmaken in de UI, zonder nieuwe code/deloys.
- **Transactieve veiligheid:** Als er halverwege rij 50 een fout optreedt, kan de hele import (met `db.transaction`) worden teruggedrolt.
- **Inzichtelijkheid:** Je kunt vooraf precies tonen: *"Er worden 2 dozen aangemaakt, 14 specimengroepen en 14 relaties."*

Belangrijke aandachtspunten voor betrouwbaarheid:

- **Lege regels/ruis:** Zoals in je voorbeeld (de `;;;` regel tussen doos 15-01 en 15-02), moet de engine lege regels automatisch kunnen negeren.
- **Caching tijdens import:** Als 100 regels achter elkaar over dezelfde "Insectendoos 15-01" gaan, wil je niet 100 keer de database bevragen. De engine moet de net-aangemaakte/gevonden `DoosId` in het geheugen vasthouden voor de duur van de import.
- **Validatie:** Per kolom moet je het datatype kunnen afdwingen (bijv. `Kolom 5` moet een geldig getal zijn).

Conclusie

Je hoeft geen specifieke code per invoer te schrijven. De beste aanpak is het bouwen van een **CSV Mapping Configuration Engine** in de nieuwe `Import` module van Kesy 2.0. Hiermee sla je per CSV-type een "profiel" op (bijv. *"Importprofiel: Insectendozen"*), waarna de generieke verwerker de CSV regel voor regel op basis van dat profiel uitvoert.

User prompt: Advies is helder. Als het handig is als CSV extra info meegegeven krijgt kunnen we dat vooraf natuurlijk meenemen om op die manier maximale flexibliteit/functionaliiteit te realiseren. Is het meegestuurd voorbeeld een goede eerste testcase?

Response: Het meegegeven voorbeeld is een **uitstekende eerste testcase**.

Het bevat namelijk precies de juiste mix van patronen en uitzonderingen om de robuustheid van je generieke import-engine grondig te testen.

Waarom dit een hele sterke testcase is

1. Het afhandelen van 1-op-N verhoudingen (Parent-Child)

- De CSV herhaalt de "Insectendoos" op elke regel, maar in je database wil je die doos slechts

één keer opzoeken of aanmaken, en daar vervolgens **meerdere** "Specimen groepen" aan koppelen. Dit dwingt de engine om slim met geheugen/caching om te gaan.

2. Samengestelde velden & Transformatieregels

- Het combineren van Kolom 3 ("Geslacht") + Kolom 4 ("Soort") tot één "Soortnaam" toont aan of je datatransformaties (zoals `CONCAT` of `JOIN_WITH_SPACE`) goed kunt configureren.

3. Gecombineerde acties op 1 object

- Een aangemaakt object moet direct op meerdere manieren verrijkt worden:
 - Een **Parameter** meekrijgen ("Toelichting" op de doos, "Aantal" op de specimengroep).
 - Een **Relatie** leggen met een vast Type-ID ("is objecttype").
 - Een **Hiërarchische Relatie** leggen met de Parent ("ligt/zit in of op").

4. Ruis & Vuile data

- De regel met `;;;;` is een perfecte praktijktest voor de **data-cleaning** van je importeur. Een goede engine moet lege regels, extra spaties (zoals achter `17.10.5 L`) en ontbrekende waarden geruisloos filteren.

Hoe je extra flexibiliteit kunt meesturen (Metadata Header)

Als je vooraf tóch extra informatie mee wilt of kunt geven, kun je werken met **Metadata in de header** van het CSV-bestand (of als JSON-sidecar).

Hiermee hoeft je in de UI niet eens handmatig het profiel te selecteren, maar herkent Kesy 2.0 de CSV direct:

```
# PROFILE: insecten_doos_import_v1
# DEFAULT_VISIBILITY: public
"Insectendoos_label";"Insectendoos_toelichting";"Geslacht";"Soort";"Aantal"
"15-01: 17.10.5 L "; "Natuurmuseum, Enschede..."; "Oxytelus";"rugosus";13
```

Conclusie

Als je generieke import-engine deze specifieke testcase succesvol en foutloos verwerkt, heb je direct **80% tot 90% van alle toekomstige CSV-importbehoeften** binnen Kesy 2.0 afgedekt.

Wanneer je zover bent om de stap naar de mapping-structuur of code te maken, kunnen we dit schema direct als vertrekpunt gebruiken.

User prompt: Laten we maar gaan beginnen zou ik willen zeggen. Metadata Header lijkt mij ideaal; daarmee kunnen we waarschijnlijk ook voorkomen dat er extra tabellen nodig zijn in database?

Response: Dat is een hele scherpe gedachte: door de profiel-en mappingregels op te slaan als een

JSON Metadata Header bovenin het CSV-bestand (of als losse `.json` profielbestanden in je code repository), hoef je inderdaad **geen extra tabellen** toe te voegen aan je SQLite database schema!

Het CSV-bestand wordt daarmee volledig zelfbeschrijvend.

Het concept: Zelfbeschrijvende CSV Header

We kunnen afspreken dat het CSV-bestand begint met één of meerdere commentaarregels (ingeleid met `#`). Alles na de `#` is een JSON-structuur waarin de mapping-logica exact staat uitgeschreven.

Voorbeeld van het CSV-bestand voor onze testcase:

```
# {
#   "profile": "Insectendoos Specimen Import",
#   "version": "1.0",
#   "delimiter": ";",
#   "ignoreEmptyRows": true,
#   "steps": [
#     {
#       "step": 1,
#       "target": "parent_object",
#       "contextKey": "DoosId",
#       "labelTemplate": "Insectendoos: {0}",
#       "objectTypeId": "019fcd20-b442-755f-af50-9cdf9716990d",
#       "parameters": [
#         { "parameterId": "Toelichting", "column": 1 }
#       ]
#     },
#     {
#       "step": 2,
#       "target": "child_object",
#       "labelTemplate": "Specimen groep: {2} {3}",
#       "objectTypeId": "019fcd20-b56f-76ae-9bf0-f98a0354b7ca",
#       "relations": [
#         { "relationTypeId": "ligt/zit in of op", "relatedContextKey": "DoosId"
#       ],
#       "parameters": [
#         { "parameterId": "Aantal", "column": 4, "type": "number" }
#       ]
#     }
#   ]
# }
# Insectendoos_label";Insectendoos_toelichting";Geslacht";Soort";Aantal
# 15-01: 17.10.5 L ";Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
# 15-01: 17.10.5 L ";Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
# ;;;;
# 15-02: 17.10.6L";Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae
```

Architectuur van de Import Module (/import)

De verwerking in Next.js bestaat uit **3 lagen**:

1. Parser & Cleaner (src/lib/csvParser.ts)

- Leest de header uit (#).
- Parst de JSON-configuratie.
- Filtert lege of ongeldige regels zoals ; ; ; en stript spaties rondom teksten (trim()).

2. In-Memory Caching & Executor (src/lib/importExecutor.ts)

- Bepaalt of een parent-object (zoals de Insectendoos) al in de lokale cache of database bestaat. Zo niet, dan maakt hij deze aan.
- Maakt het child-object aan en legt de relaties en parameters aan.

3. UI Preview & Execution Component (app/import/page.tsx)

- Toont een **Preview**: *"3 unieke dozen gevonden, 14 specimengroepen te importeren"*.
- Bevat de knop **"Voer Import Uit"**.

Eerste Stap: De Parser maken

Laten me beginnen bij de basis. We maken een helper die de CSV kan inlezen, de JSON-metadata eruit kan strippen en de regels netjes opboort.

Maak het bestand `src/lib/csvParser.ts` aan:

```
// src/lib/csvParser.ts

export interface ImportStepConfig {
  step: number;
  target: "parent_object" | "child_object";
  contextKey?: string;
  labelTemplate: string;
  objectId: string;
  typeId: string;
  relations?: {
    relationTypeId: string;
    relatedContextKey: string;
    direction: "outgoing" | "incoming";
  }[];
  parameters?: {
    parameterId: string;
    column: number;
    type?: "string" | "number";
  }[];
}
```



```
export interface ImportConfigHeader {
  profile: string;
  version: string;
  delimiter?: string;
  ignoreEmptyRows?: boolean;
  steps: ImportStepConfig[];
}

export interface ParsedCsvResult {
  config: ImportConfigHeader | null;
  headers: string[];
  rows: string[][];
  errors: string[];
}

export function parseCsvWithHeader(rawCsvText: string): ParsedCsvResult {
  const lines = rawCsvText.split(/\r?\n/);
  const jsonHeaderLines: string[] = [];
  const dataLines: string[] = [];
  const errors: string[] = [];

  // 1. Scheid JSON header regels (#) van de dataset
  for (const line of lines) {
    const trimmed = line.trim();
    if (trimmed.startsWith("#")) {
      jsonHeaderLines.push(trimmed.substring(1).trim());
    } else if (trimmed.length > 0) {
      dataLines.push(line);
    }
  }

  // 2. Parse JSON Metadata Config
  let config: ImportConfigHeader | null = null;
  if (jsonHeaderLines.length > 0) {
    try {
      config = JSON.parse(jsonHeaderLines.join("\n"));
    } catch (err: any) {
      errors.push(`Fout bij parsen van Metadata Header: ${err.message}`);
    }
  }

  const delimiter = config?.delimiter || ";";

  // 3. Verwerk data regels & filter ruis (zoals ;;;;)
  const rows: string[][] = [];
  let csvHeaders: string[] = [];

  dataLines.forEach((line, index) => {
    // Splits op scheidingsteken (rekening houdend met quotes)
```

```

const rawCols = line.split(delimiter).map((col) => col.replace(/^"(.*)"$/, "
// Check of regel leeg is (bijv. ;;;;))
const hasData = rawCols.some((col) => col.length > 0);
if (!hasData && (config?.ignoreEmptyRows ?? true)) {
  return; // Sla lege regel over
}

if (csvHeaders.length === 0) {
  csvHeaders = rawCols; // Eerste geldige regel is de CSV kolommenkop
} else {
  rows.push(rawCols);
}
});

return {
  config,
  headers: csvHeaders,
  rows,
  errors,
};
}

```

Hoe nu verder?

1. Voeg `src/lib/csvParser.ts` toe aan je project.
2. Vervolgens maken we een Server Action `verwerkCsvImport` aan die op basis van `ImportConfigHeader` de transacties naar SQLite / Drizzle uitvoert.

Zullen we de Server Action en de **Import Dashboard UI** (`/import`) gaan opzetten?

User prompt: Jazeker

Response: Laten we de Server Action en de UI-pagina voor de CSV-import instellen.

Stap 1: De Executor Server Action (`app/import/actions.ts`)

We maken een Server Action aan die de parsed data en config ontvangt en alles via Drizzle in één nette database-transactie afhandelt.

Maak `app/import/actions.ts` aan:

```

// app/import/actions.ts
"use server";

import { db } from "@src/db";
import { objects, relationValues, parameterValues } from "@src/db/schema";

```

```
import { eq } from "drizzle-orm";
import { ImportConfigHeader } from "@src/lib/csvParser";
import { revalidatePath } from "next/cache";

export async function voerCsvImportUit({
  config,
  rows,
}): {
  config: ImportConfigHeader;
  rows: string[][];
}) {
  try {
    let uniekeDozenAangemaakt = 0;
    let uniekeDozenGevonden = 0;
    let specimengroepenAangemaakt = 0;

    // In-memory cache voor parent objects gedurende deze import (bijv. DoosId's
    const contextCache: Record<string, string> = {};

    await db.transaction(async (tx) => {
      for (const row of rows) {
        // We houden per regel bij welke contextKey welk objectId heeft opgeleve
        const rowContext: Record<string, string> = {};

        for (const stepConfig of config.steps) {
          if (stepConfig.target === "parent_object") {
            // Vul placeholders in labelTemplate, bijv. "Insectendoos: {0}" -> r
            const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
            const cacheKey = `${stepConfig.contextKey}_${label}`;

            let parentId = contextCache[cacheKey];

            if (!parentId) {
              // 1. Zoek of het parent object al in de database bestaat
              const bestaand = await tx
                .select()
                .from(objects)
                .where(eq(objects.label, label))
                .get();

              if (bestaand) {
                parentId = bestaand.id;
                uniekeDozenGevonden++;
              } else {
                // 2. Zo niet: Maak nieuw parent object aan
                const newId = crypto.randomUUID();
                const now = new Date().toISOString();

                await tx.insert(objects).values({
                  id: newId,
```

```
        label: label,
        createdAt: now,
        updatedAt: now,
    });

    // Koppel "is objecttype" relatie met target
    if (stepConfig.objectTypeId) {
        await tx.insert(relationValues).values({
            id: crypto.randomUUID(),
            relationId: "is_objecttype", // of jouw type-relatie id
            sourceObjectId: stepConfig.objectTypeId,
            targetObjectId: newId,
            createdAt: now,
            updatedAt: now,
        });
    }

    // Koppel parameters als die er zijn (bijv. Toelichting)
    if (stepConfig.parameters) {
        for (const param of stepConfig.parameters) {
            const paramVal = row[param.column];
            if (paramVal) {
                await tx.insert(parameterValues).values({
                    id: crypto.randomUUID(),
                    objectId: newId,
                    parameterId: param.parameterId,
                    value: paramVal,
                    createdAt: now,
                    updatedAt: now,
                });
            }
        }
    }

    parentId = newId;
    uniekeDozenAangemaakt++;
}

// Sla op in in-memory cache
contextCache[cacheKey] = parentId;
}

if (stepConfig.contextKey) {
    rowContext[stepConfig.contextKey] = parentId;
}
} else if (stepConfig.target === "child_object") {
    // Vul child labelTemplate in, bijv. "Specimen groep: {2} {3}" -> ro
    const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
    const childId = crypto.randomUUID();
    const now = new Date().toISOString();
```

```
// 1. Maak het child object aan
await tx.insert(objects).values({
  id: childId,
  label: label,
  createdAt: now,
  updatedAt: now,
});

// 2. Leg relatie "is objecttype" voor Specimengroep
if (stepConfig.objectTypeId) {
  await tx.insert(relationValues).values({
    id: crypto.randomUUID(),
    relationId: "is_objecttype",
    sourceObjectId: stepConfig.objectTypeId,
    targetObjectId: childId,
    createdAt: now,
    updatedAt: now,
  });
}

// 3. Leg hiërarchische relaties met parent (bijv. DoosId)
if (stepConfig.relations) {
  for (const rel of stepConfig.relations) {
    const parentId = rowContext[rel.relatedContextKey] || contextCac
    if (parentId) {
      await tx.insert(relationValues).values({
        id: crypto.randomUUID(),
        relationId: rel.relationTypeId,
        sourceObjectId: rel.direction === "outgoing" ? parentId : childId,
        targetObjectId: rel.direction === "outgoing" ? childId : parentId,
        createdAt: now,
        updatedAt: now,
      });
    }
  }
}

// 4. Voeg parameters toe aan het child object (bijv. Aantal)
if (stepConfig.parameters) {
  for (const param of stepConfig.parameters) {
    const paramVal = row[param.column];
    if (paramVal) {
      await tx.insert(parameterValues).values({
        id: crypto.randomUUID(),
        objectId: childId,
        parameterId: param.parameterId,
        value: String(paramVal),
        createdAt: now,
        updatedAt: now,
      });
    }
  }
}
```

```

    });
  }
}

specimengroepenAangemaakt++;
}
}
});

revalidatePath("/");
return {
  success: true,
  stats: {
    uniekeDozenAangemaakt,
    uniekeDozenGevonden,
    specimengroepenAangemaakt,
    totaalVerwerkteRegels: rows.length,
  },
};
} catch (err: any) {
  console.error("Import execution failed:", err);
  return { success: false, error: err.message || "Fout tijdens import." };
}
}

```

Stap 2: De UI-pagina (app/import/page.tsx)

Hiermee bouwen we een overzichtelijk drag-and-drop dashboard waar je een CSV inlaadt, de preview ziet van de gedetecteerde stappen, en de import definitief uitvoert.

Maak app/import/page.tsx aan:

```

// app/import/page.tsx
"use client";

import { useState } from "react";
import { parseCsvWithHeader, ParsedCsvResult } from "@src/lib/csvParser";
import { voerCsvImportUit } from "./actions";

export default function ImportPage() {
  const [parsedData, setParsedData] = useState<ParsedCsvResult | null>(null);
  const [importing, setImporting] = useState(false);
  const [resultStats, setResultStats] = useState<any>(null);

  const handleFileUpload = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;
  }
}

```

```
const reader = new FileReader();
reader.onload = (evt) => {
  const text = evt.target?.result as string;
  if (text) {
    const result = parseCsvWithHeader(text);
    setParsedData(result);
    setResultStats(null);
  }
};
reader.readAsText(file);
};

const handleRunImport = async () => {
  if (!parsedData?.config || !parsedData.rows.length) return;

  setImporting(true);
  const res = await voerCsvImportUit({
    config: parsedData.config,
    rows: parsedData.rows,
  });
  setImporting(false);

  if (res.success) {
    setResultStats(res.stats);
  } else {
    alert(`Import mislukt: ${res.error}`);
  }
};

return (
  <div className="max-w-[1400px] mx-auto space-y-6">
    <div>
      <h1 className="text-2xl font-bold text-slate-100 flex items-center gap-2">
        <span>📁</span> Dynamische CSV Import Engine
      </h1>
      <p className="text-sm text-slate-400 mt-1">
        Laad een CSV-bestand in inclusief `#` metadata configuratie-header om
      </p>
    </div>

    { /* BESTAND UPLOADER */ }
    <div className="p-8 bg-slate-900 border-2 border-dashed border-slate-700 r">
      <input
        type="file"
        accept=".csv,.txt"
        onChange={handleFileUpload}
        className="hidden"
        id="csv-file-input"
      />
      <label htmlFor="csv-file-input" className="cursor-pointer block">
```

```

    <span className="text-3xl block mb-2">📄</span>
    <span className="text-sm font-semibold text-sky-400">Klik om CSV-besta
    <span className="text-xs text-slate-500 block mt-1">Ondersteunt .csv b
  </label>
</div>

{/* PREVIEW VAN VERWERKT BESTAND */}
{parsedData && (
  <div className="bg-slate-900 border border-slate-800 rounded-xl p-6 spac
    <div className="flex justify-between items-start border-b border-slate
      <div>
        <span className="text-xs font-mono text-sky-400 bg-sky-950 px-2 py
          Profiel: {parsedData.config?.profile || "Geen profiel header ged
        </span>
        <h2 className="text-lg font-bold text-slate-100 mt-2">
          Preview Data & Config Mapping
        </h2>
      </div>

      <button
        onClick={handleRunImport}
        disabled={importing || !parsedData.config}
        className="px-5 py-2 bg-emerald-600 hover:bg-emerald-500 text-whit
      >
        {importing ? "Bezig met importeren..." : "🚀 Voer Import Definitie
      </button>
    </div>

    {/* PARSING ERROR BANNER */}
    {parsedData.errors.length > 0 && (
      <div className="p-3 bg-rose-950/80 border border-rose-800 text-rose-
        {parsedData.errors.map((err, idx) => (
          <p key={idx}>⚠️ {err}</p>
        ))}
      </div>
    )}

    {/* IMPORT RESULTATEN STATS */}
    {resultStats && (
      <div className="p-4 bg-emerald-950/50 border border-emerald-800 roun
        <h3 className="text-sm font-bold text-emerald-400">✓ Import Succes
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4 text-xs font
          <div>
            <span className="text-slate-400 block">Dozen Aangemaakt:</span>
            <span className="text-base text-emerald-300 font-bold">{result
          </div>
          <div>
            <span className="text-slate-400 block">Dozen Hergebruikt:</spa
            <span className="text-base text-sky-300 font-bold">{resultStat

```



```

    </div>
    <div>
      <span className="text-slate-400 block">Specimengroepen:</span>
      <span className="text-base text-emerald-300 font-bold">{result
    </div>
    <div>
      <span className="text-slate-400 block">Totaal Regels:</span>
      <span className="text-base text-slate-200 font-bold">{resultSt
    </div>
  </div>
</div>
)}

{/* TABEL PREVIEW (EERSTE 5 REGELS) */}
<div className="space-y-2">
  <span className="text-xs text-slate-400 font-semibold block">
    Gedetecteerde Datarijen ({parsedData.rows.length} totaal, toont ee
  </span>
  <div className="overflow-x-auto border border-slate-800 rounded-lg">
    <table className="w-full text-xs text-left text-slate-300">
      <thead className="bg-slate-950 text-slate-400 font-mono border-b
        <tr>
          {parsedData.headers.map((h, i) => (
            <th key={i} className="p-2.5">{h}</th>
          ))}
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-800/60 font-mono bg-slat
        {parsedData.rows.slice(0, 5).map((row, rIdx) => (
          <tr key={rIdx} className="hover:bg-slate-800/40">
            {row.map((col, cIdx) => (
              <td key={cIdx} className="p-2.5 truncate max-w-[200px]">
            ))}
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</div>
</div>
)}
</div>
);
}

```

User prompt: Ik zie een paar zaken verkeerd gaan: app maakt overal gebruik van UUIDv7; handig als ik schema ook weer even meestuur (en repository denk ik)

Response: Bedankt voor het meesturen van het schema en de repository. Dit brengt meteen **drie cruciale punten** aan het licht waar de eerdere Server Action zou zijn vastgelopen:

1. **UUIDv7**: In plaats van `crypto.randomUUID()` moeten we `v7` as `uuidv7` uit de `uuid` package gebruiken.

2. Database Veldnamen & Verplichte Velden:

- `validFrom`: Verplicht ISO-timestamp string veld op `objects`, `relationValues` en `parameterValues`.
- `relationValues`: Velden heten `sourceId` en `targetId` (niet `sourceObjectId`/`targetObjectId`).
- `parameterValues`: Bevat `targetId` en `targetType` (bijv. `'object'`). `objectId` bestaat niet.

3. **Database Import**: In jouw repository gebruik je `import { activeDb as db } from "@src/lib/db"`.

Hier zijn de gecorrigeerde bestanden afgestemd op jouw exacte schema.

1. Aangepaste CSV Parser (`src/lib/csvParser.ts`)

Zorg ervoor dat het CSV-header voorbeeld exact aansluit bij de geldige IDs en velden uit jouw database:

```
// src/lib/csvParser.ts

export interface ImportStepConfig {
  step: number;
  target: "parent_object" | "child_object";
  contextKey?: string;
  labelTemplate: string;
  objectTypeId: string;
  relations?: {
    relationTypeId: string;
    relatedContextKey: string;
    direction: "outgoing" | "incoming";
  }[];
  parameters?: {
    parameterId: string;
    column: number;
  }[];
}

export interface ImportConfigHeader {
  profile: string;
  version: string;
  delimiter?: string;
  ignoreEmptyRows?: boolean;
```

```
    steps: ImportStepConfig[];
  }

export interface ParsedCsvResult {
  config: ImportConfigHeader | null;
  headers: string[];
  rows: string[][];
  errors: string[];
}

export function parseCsvWithHeader(rawCsvText: string): ParsedCsvResult {
  const lines = rawCsvText.split(/\r?\n/);
  const jsonHeaderLines: string[] = [];
  const dataLines: string[] = [];
  const errors: string[] = [];

  for (const line of lines) {
    const trimmed = line.trim();
    if (trimmed.startsWith("#")) {
      jsonHeaderLines.push(trimmed.substring(1).trim());
    } else if (trimmed.length > 0) {
      dataLines.push(line);
    }
  }

  let config: ImportConfigHeader | null = null;
  if (jsonHeaderLines.length > 0) {
    try {
      config = JSON.parse(jsonHeaderLines.join("\n"));
    } catch (err: any) {
      errors.push(`Fout bij parsen van Metadata Header: ${err.message}`);
    }
  }

  const delimiter = config?.delimiter || ";";
  const rows: string[][] = [];
  let csvHeaders: string[] = [];

  dataLines.forEach((line) => {
    const rawCols = line.split(delimiter).map((col) => col.replace(/^"(.*)"$/, " "

    const hasData = rawCols.some((col) => col.length > 0);
    if (!hasData && (config?.ignoreEmptyRows ?? true)) {
      return;
    }

    if (csvHeaders.length === 0) {
      csvHeaders = rawCols;
    } else {
      rows.push(rawCols);
    }
  });
}
```

```

    }
  });

  return {
    config,
    headers: csvHeaders,
    rows,
    errors,
  };
}

```

2. Aangepaste Server Action (app/import/actions.ts)

Volledig afgestemd op **UUIDv7**, `activeDb`, verplichte `validFrom` velden en correcte kolomnamen.

```

// app/import/actions.ts
"use server";

import { activeDb as db } from "@src/lib/db";
import * as schema from "@src/db/schema";
import { eq } from "drizzle-orm";
import { ImportConfigHeader } from "@src/lib/csvParser";
import { revalidatePath } from "next/cache";
import { v7 as uuidv7 } from "uuid";

export async function voerCsvImportUit({
  config,
  rows,
}: {
  config: ImportConfigHeader;
  rows: string[][];
}) {
  try {
    let uniekeDozenAangemaakt = 0;
    let uniekeDozenGevonden = 0;
    let specimengroepenAangemaakt = 0;

    const contextCache: Record<string, string> = {};

    await db.transaction(async (tx) => {
      const nu = new Date().toISOString();

      for (const row of rows) {
        const rowContext: Record<string, string> = {};

        for (const stepConfig of config.steps) {
          if (stepConfig.target === "parent_object") {
            const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind) => {
              const cacheKey = `${stepConfig.contextKey}_${ind}`;

```

```
let parentId = contextCache[cacheKey];

if (!parentId) {
  const [bestaand] = await tx
    .select()
    .from(schema.objects)
    .where(eq(schema.objects.label, label));

  if (bestaand) {
    parentId = bestaand.id;
    uniekeDozenGevonden++;
  } else {
    parentId = uuidv7();

    await tx.insert(schema.objects).values({
      id: parentId,
      label: label,
      isConfidential: false,
      validFrom: nu,
      updatedAt: nu,
    });

    // Koppel "is objecttype" relatie
    if (stepConfig.objectTypeId) {
      await tx.insert(schema.relationValues).values({
        id: uuidv7(),
        relationId: stepConfig.objectTypeId,
        sourceId: parentId,
        targetId: stepConfig.objectTypeId,
        isConfidential: false,
        validFrom: nu,
        updatedAt: nu,
      });
    }
  }

  // Koppel parameters
  if (stepConfig.parameters) {
    for (const param of stepConfig.parameters) {
      const paramVal = row[param.column];
      if (paramVal) {
        await tx.insert(schema.parameterValues).values({
          id: uuidv7(),
          parameterId: param.parameterId,
          targetId: parentId,
          targetType: "object",
          value: paramVal,
          isConfidential: false,
          validFrom: nu,
          updatedAt: nu,
        });
      }
    }
  }
}
```

```
        }
      }
    }

    uniekeDozenAangemaakt++;
  }

  contextCache[cacheKey] = parentId;
}

if (stepConfig.contextKey) {
  rowContext[stepConfig.contextKey] = parentId;
}
} else if (stepConfig.target === "child_object") {
  const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
  const childId = uuidv7());

  await tx.insert(schema.objects).values({
    id: childId,
    label: label,
    isConfidential: false,
    validFrom: nu,
    updatedAt: nu,
  });

  // Koppel "is objecttype" relatie
  if (stepConfig.objectTypeId) {
    await tx.insert(schema.relationValues).values({
      id: uuidv7(),
      relationId: stepConfig.objectTypeId,
      sourceId: childId,
      targetId: stepConfig.objectTypeId,
      isConfidential: false,
      validFrom: nu,
      updatedAt: nu,
    });
  }

  // Koppel hiërarchische relaties met parent (bijv. DoosId)
  if (stepConfig.relations) {
    for (const rel of stepConfig.relations) {
      const parentId = rowContext[rel.relatedContextKey] || contextCac
      if (parentId) {
        await tx.insert(schema.relationValues).values({
          id: uuidv7(),
          relationId: rel.relationTypeId,
          sourceId: rel.direction === "outgoing" ? parentId : childId,
          targetId: rel.direction === "outgoing" ? childId : parentId,
          isConfidential: false,
          validFrom: nu,
```

```

                updatedAt: nu,
            });
        }
    }
}

// Voeg parameters toe
if (stepConfig.parameters) {
    for (const param of stepConfig.parameters) {
        const paramVal = row[param.column];
        if (paramVal) {
            await tx.insert(schema.parameterValues).values({
                id: uuidv7(),
                parameterId: param.parameterId,
                targetId: childId,
                targetType: "object",
                value: String(paramVal),
                isConfidential: false,
                validFrom: nu,
                updatedAt: nu,
            });
        }
    }
}

specimengroepenAangemaakt++;
}
}
});

revalidatePath("/");
return {
    success: true,
    stats: {
        uniekeDozenAangemaakt,
        uniekeDozenGevonden,
        specimengroepenAangemaakt,
        totaalVerwerkteRegels: rows.length,
    },
};
} catch (err: any) {
    console.error("Import execution failed:", err);
    return { success: false, error: err.message || "Fout tijdens import." };
}
}

```

3. CSV Invoerbestand met geldige Metadata Header

Het test-CSV bestand met de juiste ID's uit je database:

```
# {
#   "profile": "Insectendoos Specimen Import",
#   "version": "1.0",
#   "delimiter": ";",
#   "ignoreEmptyRows": true,
#   "steps": [
#     {
#       "step": 1,
#       "target": "parent_object",
#       "contextKey": "DoosId",
#       "labelTemplate": "Insectendoos: {0}",
#       "objectTypeId": "019fcd20-b442-755f-af50-9cdf9716990d",
#       "parameters": [
#         { "parameterId": "Toelichting", "column": 1 }
#       ]
#     },
#     {
#       "step": 2,
#       "target": "child_object",
#       "labelTemplate": "Specimen groep: {2} {3}",
#       "objectTypeId": "019fcd20-b56f-76ae-9bf0-f98a0354b7ca",
#       "relations": [
#         { "relationTypeId": "relatie_zit_in", "relatedContextKey": "DoosId", "
#       ],
#       "parameters": [
#         { "parameterId": "Aantal", "column": 4 }
#       ]
#     }
#   ]
# }
# Insectendoos_label";"Insectendoos_toelichting";"Geslacht";"Soort";"Aantal"
# "15-01: 17.10.5 L ";;"Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
# "15-01: 17.10.5 L ";;"Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
# "15-01: 17.10.5 L ";;"Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
# ;;;;
# "15-02: 17.10.6L";;"Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae
```

User prompt: Extra realiseer ik me: bij objecten moet duidelijk zijn of het vertrouwelijke of publieke objecten zijn. Relatie is vertrouwelijk als één (of beide) object vertrouwelijk is en parameter_waarden zijn vertrouwelijk als object vertrouwelijk is. Voor objecten kan vertrouwelijk/publiek voor hele csv identiek verondersteld worden.

Response: Dat is een heel belangrijk inzicht dat strak aansluit bij je schema! In `schema.ts` hebben `objects`, `relationValues` en `parameterValues` namelijk allemaal het veld `isConfidential` (boolean).

Met deze specifieke vertrouwelijkheidsregels kunnen we dit elegant inbouwen:

1. **CSV-niveau:** We voegen een optionele instelling `isConfidential` toe aan de metadata-header (standaard `false`).

2. Afgeleide Vertrouwelijkheid:

- **Object:** Neemt `isConfidential` van de import-instelling over.
- **ParameterValue:** Wordt vertrouwelijk als het bijbehorende object vertrouwelijk is (`isConfidential = parentIsConfidential`).
- **RelationValue:** Wordt vertrouwelijk als **source OF target** (of beiden) vertrouwelijk is (`isConfidential = sourceIsConfidential || targetIsConfidential`).

1. Update CSV Parser Interface (`src/lib/csvParser.ts`)

Voeg `isConfidential` toe aan de header interface:

```
export interface ImportConfigHeader {
  profile: string;
  version: string;
  delimiter?: string;
  ignoreEmptyRows?: boolean;
  isConfidential?: boolean; // 🔒 Nieuw: bepaalt vertrouwelijkheid van de hele import
  steps: ImportStepConfig[];
}
```

2. Aangepaste Server Action (`app/import/actions.ts`)

Hier verwerken we de afgeleide logica voor objecten, parameters en relaties:

```
// app/import/actions.ts
"use server";

import { activeDb as db } from "@src/lib/db";
import * as schema from "@src/db/schema";
import { eq, inArray } from "drizzle-orm";
import { ImportConfigHeader } from "@src/lib/csvParser";
import { revalidatePath } from "next/cache";
import { v7 as uuidv7 } from "uuid";

export async function voerCsvImportUit({
  config,
  rows,
}: {
  config: ImportConfigHeader;
  rows: string[][];
}) {
  try {
    let uniekeDozenAangemaakt = 0;
    let uniekeDozenGevonden = 0;
```

```
let specimengroepenAangemaakt = 0;

// 🗝 Bepaal basis vertrouwelijkheid uit header (standaard false)
const importIsConfidential = Boolean(config.isConfidential);

// Cache voor context keys: bewaart zowel de ID als de vertrouwelijkheid van
const contextCache: Record<string, { id: string; isConfidential: boolean }>

await db.transaction(async (tx) => {
  const nu = new Date().toISOString();

  for (const row of rows) {
    const rowContext: Record<string, { id: string; isConfidential: boolean }

    for (const stepConfig of config.steps) {
      if (stepConfig.target === "parent_object") {
        const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
        const cacheKey = `${stepConfig.contextKey}_${label}`;

        let parentInfo = contextCache[cacheKey];

        if (!parentInfo) {
          const [bestaand] = await tx
            .select()
            .from(schema.objects)
            .where(eq(schema.objects.label, label));

          if (bestaand) {
            parentInfo = { id: bestaand.id, isConfidential: bestaand.isConfi
            uniekeDozenGevonden++;
          } else {
            const newParentId = uuidv7();

            // 1. Maak Parent Object aan
            await tx.insert(schema.objects).values({
              id: newParentId,
              label: label,
              isConfidential: importIsConfidential, // 🗝
              validFrom: nu,
              updatedAt: nu,
            });

            // 2. Koppel "is objecttype" relatie
            // Relatie is vertrouwelijk als het object vertrouwelijk is
            if (stepConfig.objectTypeId) {
              await tx.insert(schema.relationValues).values({
                id: uuidv7(),
                relationId: stepConfig.objectTypeId,
                sourceId: newParentId,
                targetId: stepConfig.objectTypeId,
```

```
        isConfidential: importIsConfidential, // 🔒
        validFrom: nu,
        updatedAt: nu,
    }));
}

// 3. Koppel parameters (bijv. Toelichting)
if (stepConfig.parameters) {
    for (const param of stepConfig.parameters) {
        const paramVal = row[param.column];
        if (paramVal) {
            await tx.insert(schema.parameterValues).values({
                id: uuidv7(),
                parameterId: param.parameterId,
                targetId: newParentId,
                targetType: "object",
                value: paramVal,
                isConfidential: importIsConfidential, // 🔒 Parameter on
                validFrom: nu,
                updatedAt: nu,
            });
        }
    }
}

parentInfo = { id: newParentId, isConfidential: importIsConfiden
uniekeDozenAangemaakt++;
}

contextCache[cacheKey] = parentInfo;
}

if (stepConfig.contextKey) {
    rowContext[stepConfig.contextKey] = parentInfo;
}
} else if (stepConfig.target === "child_object") {
    const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
    const childId = uuidv7();
    const childIsConfidential = importIsConfidential; // 🔒

    // 1. Maak Child Object aan
    await tx.insert(schema.objects).values({
        id: childId,
        label: label,
        isConfidential: childIsConfidential, // 🔒
        validFrom: nu,
        updatedAt: nu,
    });
}
```

```

// 2. Koppel "is objecttype" relatie
if (stepConfig.objectTypeId) {
  await tx.insert(schema.relationValues).values({
    id: uuidv7(),
    relationId: stepConfig.objectTypeId,
    sourceId: childId,
    targetId: stepConfig.objectTypeId,
    isConfidential: childIsConfidential, // 🔒
    validFrom: nu,
    updatedAt: nu,
  });
}

// 3. Koppel hiërarchische relaties met parent
if (stepConfig.relations) {
  for (const rel of stepConfig.relations) {
    const parentInfo = rowContext[rel.relatedContextKey] || contextC
    if (parentInfo) {
      const sourceId = rel.direction === "outgoing" ? parentInfo.id
      const targetId = rel.direction === "outgoing" ? childId : parentInfo.id

      // 🔒 REGEL: Relatie is vertrouwelijk als SOURCE OF TARGET vertrouwelijk is
      const relationIsConfidential = parentInfo.isConfidential || childIsConfidential

      await tx.insert(schema.relationValues).values({
        id: uuidv7(),
        relationId: rel.relationTypeId,
        sourceId: sourceId,
        targetId: targetId,
        isConfidential: relationIsConfidential, // 🔒
        validFrom: nu,
        updatedAt: nu,
      });
    }
  }
}

// 4. Voeg parameters toe (bijv. Aantal)
if (stepConfig.parameters) {
  for (const param of stepConfig.parameters) {
    const paramVal = row[param.column];
    if (paramVal) {
      await tx.insert(schema.parameterValues).values({
        id: uuidv7(),
        parameterId: param.parameterId,
        targetId: childId,
        targetType: "object",
        value: String(paramVal),
        isConfidential: childIsConfidential, // 🔒 Parameter overnemen
      });
    }
  }
}

```

```

        validFrom: nu,
        updatedAt: nu,
    });
    }
}

specimengroepenAangemaakt++;
}
}
});

revalidatePath("/");
return {
    success: true,
    stats: {
        uniekeDozenAangemaakt,
        uniekeDozenGevonden,
        specimengroepenAangemaakt,
        totaalVerwerkteRegels: rows.length,
        isConfidential: importIsConfidential,
    },
};
} catch (err: any) {
    console.error("Import execution failed:", err);
    return { success: false, error: err.message || "Fout tijdens import." };
}
}

```

3. Voorbeeld Header met Vertrouwelijkheid

In de CSV kun je `isConfidential` nu eenvoudig op `true` of `false` zetten:

```

# {
#   "profile": "Insectendoos Specimen Import",
#   "version": "1.0",
#   "delimiter": ";",
#   "ignoreEmptyRows": true,
#   "isConfidential": false,
#   "steps": [ ... ]
# }

```

Als een doos in een eerdere import als **vertrouwelijk** (`isConfidential = true`) was aangemaakt, en je importeert er later een **publieke** specimengroep bij (`isConfidential = false`), dan zal de tussenliggende relatie automatisch op **vertrouwelijk** SPRINGEN door de check `parentInfo.isConfidential || childIsConfidential`. Precies zoals je bedoelde!

User prompt: [{ "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "7006", "severity": 8, "message": "Parameter 'tx' implicitly has an 'any' type.", "source": "ts", "startLineNumber": 29, "startColumn": 33, "endLineNumber": 29, "endColumn": 35, "modelVersionId": 3, "origin": "extHost1" }]

Response: Deze TypeScript-fout ontstaat doordat TypeScript het type van de transactie-variabele `tx` binnen `db.transaction()` niet automatisch kan afleiden wanneer Drizzle gebruikt wordt met een SQLite-driver.

In Drizzle ORM kun je dit eenvoudig oplossen door het type expliciet mee te geven aan de transactie-callback.

Oplossing

Voeg het juiste type voor de transactie toe bovenin `app/import/actions.ts`:

```
// app/import/actions.ts
"use server";

import { activeDb as db } from "@src/lib/db";
import * as schema from "@src/db/schema";
import { eq } from "drizzle-orm";
import { ImportConfigHeader } from "@src/lib/csvParser";
import { revalidatePath } from "next/cache";
import { v7 as uuidv7 } from "uuid";

// 1. Haal het juiste transactie-type op uit het activeDb object
type Transaction = Parameters<Parameters<typeof db.transaction>[0]>[0]>;

export async function voerCsvImportUit({
  config,
  rows,
}: {
  config: ImportConfigHeader;
  rows: string[][];
}) {
  try {
    let uniekeDozenAangemaakt = 0;
    let uniekeDozenGevonden = 0;
    let specimengroepenAangemaakt = 0;

    const importIsConfidential = Boolean(config.isConfidential);
    const contextCache: Record<string, { id: string; isConfidential: boolean }>

    // 2. Geef het 'Transaction' type mee aan 'tx'
    await db.transaction(async (tx: Transaction) => {
      const nu = new Date().toISOString();

      for (const row of rows) {
        const rowContext: Record<string, { id: string; isConfidential: boolean }>
```

```
for (const stepConfig of config.steps) {
  if (stepConfig.target === "parent_object") {
    const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
    const cacheKey = `${stepConfig.contextKey}_${label}`;

    let parentInfo = contextCache[cacheKey];

    if (!parentInfo) {
      const [bestaand] = await tx
        .select()
        .from(schema.objects)
        .where(eq(schema.objects.label, label));

      if (bestaand) {
        parentInfo = { id: bestaand.id, isConfidential: bestaand.isConfi
        uniekeDozenGevonden++;
      } else {
        const newParentId = uuidv7();

        await tx.insert(schema.objects).values({
          id: newParentId,
          label: label,
          isConfidential: importIsConfidential,
          validFrom: nu,
          updatedAt: nu,
        });

        if (stepConfig.objectTypeId) {
          await tx.insert(schema.relationValues).values({
            id: uuidv7(),
            relationId: stepConfig.objectTypeId,
            sourceId: newParentId,
            targetId: stepConfig.objectTypeId,
            isConfidential: importIsConfidential,
            validFrom: nu,
            updatedAt: nu,
          });
        }
      }

      if (stepConfig.parameters) {
        for (const param of stepConfig.parameters) {
          const paramVal = row[param.column];
          if (paramVal) {
            await tx.insert(schema.parameterValues).values({
              id: uuidv7(),
              parameterId: param.parameterId,
              targetId: newParentId,
              targetType: "object",
              value: paramVal,
```

```

        isConfidential: importIsConfidential,
        validFrom: nu,
        updatedAt: nu,
    });
    }
}

parentInfo = { id: newParentId, isConfidential: importIsConfiden
uniekeDozenAangemaakt++;
}

contextCache[cacheKey] = parentInfo;
}

if (stepConfig.contextKey) {
    rowContext[stepConfig.contextKey] = parentInfo;
}
} else if (stepConfig.target === "child_object") {
    const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
    const childId = uuidv7();
    const childIsConfidential = importIsConfidential;

    await tx.insert(schema.objects).values({
        id: childId,
        label: label,
        isConfidential: childIsConfidential,
        validFrom: nu,
        updatedAt: nu,
    });

    if (stepConfig.objectTypeId) {
        await tx.insert(schema.relationValues).values({
            id: uuidv7(),
            relationId: stepConfig.objectTypeId,
            sourceId: childId,
            targetId: stepConfig.objectTypeId,
            isConfidential: childIsConfidential,
            validFrom: nu,
            updatedAt: nu,
        });
    }

    if (stepConfig.relations) {
        for (const rel of stepConfig.relations) {
            const parentInfo = rowContext[rel.relatedContextKey] || contextC
            if (parentInfo) {
                const sourceId = rel.direction === "outgoing" ? parentInfo.id
                const targetId = rel.direction === "outgoing" ? childId : pare
                const relationIsConfidential = parentInfo.isConfidential || ch

```



```
        await tx.insert(schema.relationValues).values({
            id: uuidv7(),
            relationId: rel.relationTypeId,
            sourceId: sourceId,
            targetId: targetId,
            isConfidential: relationIsConfidential,
            validFrom: nu,
            updatedAt: nu,
        });
    }
}

if (stepConfig.parameters) {
    for (const param of stepConfig.parameters) {
        const paramVal = row[param.column];
        if (paramVal) {
            await tx.insert(schema.parameterValues).values({
                id: uuidv7(),
                parameterId: param.parameterId,
                targetId: childId,
                targetType: "object",
                value: String(paramVal),
                isConfidential: childIsConfidential,
                validFrom: nu,
                updatedAt: nu,
            });
        }
    }
}

specimengroepenAangemaakt++;
}
}
});

revalidatePath("/");
return {
    success: true,
    stats: {
        uniekeDozenAangemaakt,
        uniekeDozenGevonden,
        specimengroepenAangemaakt,
        totaalVerwerkteRegels: rows.length,
        isConfidential: importIsConfidential,
    },
};
} catch (err: any) {
```

```

    console.error("Import execution failed:", err);
    return { success: false, error: err.message || "Fout tijdens import." };
  }
}

```

Met de regel `type Transaction = Parameters<Parameters<typeof db.transaction>[0]>[0]`; wordt het exacte transactie-type automatisch afgeleid van jouw specifieke Drizzle-database instance (`activeDb`), waardoor de TypeScript-fout verdwijnt.

User prompt: [{ "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2344", "severity": 8, "message": "Type 'unknown' does not satisfy the constraint '(...args: any) => any'.", "source": "ts", "startLineNumber": 12, "startColumn": 31, "endLineNumber": 12, "endColumn": 67, "modelVersionId": 4, "origin": "extHost1"}, { "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2339", "severity": 8, "message": "Property 'select' does not exist on type 'never'.", "source": "ts", "startLineNumber": 45, "startColumn": 18, "endLineNumber": 45, "endColumn": 24, "modelVersionId": 4, "origin": "extHost1"}, { "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2339", "severity": 8, "message": "Property 'insert' does not exist on type 'never'.", "source": "ts", "startLineNumber": 55, "startColumn": 26, "endLineNumber": 55, "endColumn": 32, "modelVersionId": 4, "origin": "extHost1"}, { "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2339", "severity": 8, "message": "Property 'insert' does not exist on type 'never'.", "source": "ts", "startLineNumber": 64, "startColumn": 28, "endLineNumber": 64, "endColumn": 34, "modelVersionId": 4, "origin": "extHost1"}, { "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2339", "severity": 8, "message": "Property 'insert' does not exist on type 'never'.", "source": "ts", "startLineNumber": 79, "startColumn": 32, "endLineNumber": 79, "endColumn": 38, "modelVersionId": 4, "origin": "extHost1"}, { "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2339", "severity": 8, "message": "Property 'insert' does not exist on type 'never'.", "source": "ts", "startLineNumber": 108, "startColumn": 22, "endLineNumber": 108, "endColumn": 28, "modelVersionId": 4, "origin": "extHost1"}, { "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2339", "severity": 8, "message": "Property 'insert' does not exist on type 'never'.", "source": "ts", "startLineNumber": 117, "startColumn": 24, "endLineNumber": 117, "endColumn": 30, "modelVersionId": 4, "origin": "extHost1"}, { "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2339", "severity": 8, "message": "Property 'insert' does not exist on type 'never'.", "source": "ts", "startLineNumber": 136, "startColumn": 28, "endLineNumber": 136, "endColumn": 34, "modelVersionId": 4, "origin": "extHost1"}, { "resource": "/C:/TB-Projecten/toolbox/kesy-2.0/app/import/actions.ts", "owner": "typescript", "code": "2339", "severity": 8, "message": "Property 'insert' does not exist on type 'never'.", "source": "ts", "startLineNumber": 153, "startColumn": 28, "endLineNumber": 153, "endColumn": 34, "modelVersionId": 4, "origin": "extHost1"}]

Response: De fouten ontstaan doordat `Parameters<typeof db.transaction>` degradeert naar `unknown` als Drizzle 'db' is geef als een abstracte/dynamische database instance. Daardoor wordt `tx` als `never` getypeerd.

In plaats van de helper-type truc kun je `tx` simpelweg typeren als `typeof db` (of `any`). Drizzle's `tx` object implementeert dezelfde API als het `db` object zelf (`select`, `insert`, etc.).

Vervang de bovenkant en de transactie-call in `app/import/actions.ts`:

```
// app/import/actions.ts
"use server";

import { activeDb as db } from "@src/lib/db";
import * as schema from "@src/db/schema";
import { eq } from "drizzle-orm";
import { ImportConfigHeader } from "@src/lib/csvParser";
import { revalidatePath } from "next/cache";
import { v7 as uuidv7 } from "uuid";

export async function voerCsvImportUit({
  config,
  rows,
}: {
  config: ImportConfigHeader;
  rows: string[][];
}) {
  try {
    let uniekeDozenAangemaakt = 0;
    let uniekeDozenGevonden = 0;
    let specimengroepenAangemaakt = 0;

    const importIsConfidential = Boolean(config.isConfidential);
    const contextCache: Record<string, { id: string; isConfidential: boolean }>

    // Cast tx expliciet naar typeof db om TS te voorzien van alle Drizzle metho
    await db.transaction(async (tx: any) => {
      const nu = new Date().toISOString();

      for (const row of rows) {
        const rowContext: Record<string, { id: string; isConfidential: boolean }>

        for (const stepConfig of config.steps) {
          if (stepConfig.target === "parent_object") {
            const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
            const cacheKey = `${stepConfig.contextKey}_${label}`;

            let parentInfo = contextCache[cacheKey];

            if (!parentInfo) {
              const [bestaand] = await tx
                .select()
                .from(schema.objects)
                .where(eq(schema.objects.label, label));
```

```
if (bestaand) {
  parentInfo = { id: bestaand.id, isConfidential: bestaand.isConfi
  uniekeDozenGevonden++;
} else {
  const newParentId = uuidv7();

  await tx.insert(schema.objects).values({
    id: newParentId,
    label: label,
    isConfidential: importIsConfidential,
    validFrom: nu,
    updatedAt: nu,
  });

  if (stepConfig.objectTypeId) {
    await tx.insert(schema.relationValues).values({
      id: uuidv7(),
      relationId: stepConfig.objectTypeId,
      sourceId: newParentId,
      targetId: stepConfig.objectTypeId,
      isConfidential: importIsConfidential,
      validFrom: nu,
      updatedAt: nu,
    });
  }

  if (stepConfig.parameters) {
    for (const param of stepConfig.parameters) {
      const paramVal = row[param.column];
      if (paramVal) {
        await tx.insert(schema.parameterValues).values({
          id: uuidv7(),
          parameterId: param.parameterId,
          targetId: newParentId,
          targetType: "object",
          value: paramVal,
          isConfidential: importIsConfidential,
          validFrom: nu,
          updatedAt: nu,
        });
      }
    }
  }

  parentInfo = { id: newParentId, isConfidential: importIsConfiden
  uniekeDozenAangemaakt++;
}

contextCache[cacheKey] = parentInfo;
}
```

```

    if (stepConfig.contextKey) {
      rowContext[stepConfig.contextKey] = parentInfo;
    }
  } else if (stepConfig.target === "child_object") {
    const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
    const childId = uuidv7();
    const childIsConfidential = importIsConfidential;

    await tx.insert(schema.objects).values({
      id: childId,
      label: label,
      isConfidential: childIsConfidential,
      validFrom: nu,
      updatedAt: nu,
    });

    if (stepConfig.objectTypeId) {
      await tx.insert(schema.relationValues).values({
        id: uuidv7(),
        relationId: stepConfig.objectTypeId,
        sourceId: childId,
        targetId: stepConfig.objectTypeId,
        isConfidential: childIsConfidential,
        validFrom: nu,
        updatedAt: nu,
      });
    }
  }

  if (stepConfig.relations) {
    for (const rel of stepConfig.relations) {
      const parentInfo = rowContext[rel.relatedContextKey] || contextC
      if (parentInfo) {
        const sourceId = rel.direction === "outgoing" ? parentInfo.id
        const targetId = rel.direction === "outgoing" ? childId : pare
        const relationIsConfidential = parentInfo.isConfidential || ch

        await tx.insert(schema.relationValues).values({
          id: uuidv7(),
          relationId: rel.relationTypeId,
          sourceId: sourceId,
          targetId: targetId,
          isConfidential: relationIsConfidential,
          validFrom: nu,
          updatedAt: nu,
        });
      }
    }
  }
}

```

```

    if (stepConfig.parameters) {
      for (const param of stepConfig.parameters) {
        const paramVal = row[param.column];
        if (paramVal) {
          await tx.insert(schema.parameterValues).values({
            id: uuidv7(),
            parameterId: param.parameterId,
            targetId: childId,
            targetType: "object",
            value: String(paramVal),
            isConfidential: childIsConfidential,
            validFrom: nu,
            updatedAt: nu,
          });
        }
      }
    }

    specimengroepenAangemaakt++;
  }
}
});

revalidatePath("/");
return {
  success: true,
  stats: {
    uniekeDozenAangemaakt,
    uniekeDozenGevonden,
    specimengroepenAangemaakt,
    totaalVerwerkteRegels: rows.length,
    isConfidential: importIsConfidential,
  },
};
} catch (err: any) {
  console.error("Import execution failed:", err);
  return { success: false, error: err.message || "Fout tijdens import." };
}
}

```

User prompt: Ik zie dat ik een aantal Id's nog niet heb aangegeven: relatietype "is objecttype" heeft Id: 019fcdd3-721a-7512-b755-cddd67f43eb6 parameter "Toelichting" heeft Id: 019fc74c-cf8c-74ff-a3b6-b6d21c651a19 relatie "ligt/zit in of op" heeft Id: 019fad01-ca30-769d-8c9c-6fc70fa9db0a parameter "Aantal" heeft Id: 019fad6f-c149-7795-b3e5-751c8b2b7949

Response: Met deze specifieke UUIDv7 identifiers is de configuratie voor de "Insectendoos" testcase nu 100% compleet en direct bruikbaar.

Hier is de definitieve **Metadata Header** voor in je CSV-bestand met alle juiste ID's op hun plek:

```
# {
#   "profile": "Insectendoos Specimen Import",
#   "version": "1.0",
#   "delimiter": ";",
#   "ignoreEmptyRows": true,
#   "isConfidential": false,
#   "steps": [
#     {
#       "step": 1,
#       "target": "parent_object",
#       "contextKey": "DoosId",
#       "labelTemplate": "Insectendoos: {0}",
#       "objectTypeId": "019fcdd3-721a-7512-b755-cddd67f43eb6",
#       "parameters": [
#         { "parameterId": "019fc74c-cf8c-74ff-a3b6-b6d21c651a19", "column": 1 }
#       ]
#     },
#     {
#       "step": 2,
#       "target": "child_object",
#       "labelTemplate": "Specimen groep: {2} {3}",
#       "objectTypeId": "019fcdd3-721a-7512-b755-cddd67f43eb6",
#       "relations": [
#         {
#           "relationTypeId": "019fad01-ca30-769d-8c9c-6fc70fa9db0a",
#           "relatedContextKey": "DoosId",
#           "direction": "outgoing"
#         }
#       ],
#       "parameters": [
#         { "parameterId": "019fad6f-c149-7795-b3e5-751c8b2b7949", "column": 4 }
#       ]
#     }
#   ]
# }
# "Insectendoos_label";"Insectendoos_toelichting";"Geslacht";"Soort";"Aantal"
# "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
# "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
# "15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
# ;;;;
# "15-02: 17.10.6L"; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae
```

Overzicht van de verwerking:

- **Stap 1 (Parent):** Maakt/zoekt de Insectendoos met de "is objecttype" relatie (019fcdd3...) en voegt de parameter **Toelichting** (019fc74c...) toe op basis van kolom index 1.
- **Stap 2 (Child):** Maakt de Specimengroep aan met de "is objecttype" relatie (019fcdd3...),

legt de relatie **ligt/zit in of op** (019fad01...) naar de Parent-doos, en kent de parameter **Aantal** (019fad6f...) toe op basis van kolom index 4.

Alles staat nu klaar om de eerste testimport in de UI op `/import` uit te voeren!

User prompt: Ik mis een ingaande relatie bij de nieuwe doos: met dit object als target een relatie "is objecttype" leggen met de source "Object type: Insectendoos" (ID: 019fcd20-b442-755f-af50-9cdf9716990d) relatietype "is objecttype" heeft Id:019fcdd3-721a-7512-b755-cddd67f43eb6

Response: Scherp opgemerkt! In de eerdere code werd `stepConfig.objectTypeId` als target gebruikt, terwijl jouw schema vereist dat de relatie **"is objecttype"** (019fcdd3-721a-7512-b755-cddd67f43eb6) als volgt gelegd wordt:

- **Source:** Het specifieke Object Type (bijv. "Object type: Insectendoos" met ID 019fcd20-b442-755f-af50-9cdf9716990d).
- **Target:** Het nieuw aangemaakte object (de Doos of Specimengroep).
- **RelationId:** De relatie-definitie "is objecttype" (019fcdd3-721a-7512-b755-cddd67f43eb6).

Aangepaste Server Action (`app/import/actions.ts`)

Hier is het gecorrigeerde blok voor de `relationValues` insert voor zowel het parent- als het child-object:

```
// app/import/actions.ts
"use server";

import { activeDb as db } from "@src/lib/db";
import * as schema from "@src/db/schema";
import { eq } from "drizzle-orm";
import { ImportConfigHeader } from "@src/lib/csvParser";
import { revalidatePath } from "next/cache";
import { v7 as uuidv7 } from "uuid";

// ID van het relatietype "is objecttype"
const IS_OBJECTTYPE_RELATION_ID = "019fcdd3-721a-7512-b755-cddd67f43eb6";

export async function voerCsvImportUit({
  config,
  rows,
}: {
  config: ImportConfigHeader;
  rows: string[][];
}) {
  try {
    let uniekeDozenAangemaakt = 0;
```



```
let uniekeDozenGevonden = 0;
let specimengroepenAangemaakt = 0;

const importIsConfidential = Boolean(config.isConfidential);
const contextCache: Record<string, { id: string; isConfidential: boolean }>

await db.transaction(async (tx: any) => {
  const nu = new Date().toISOString();

  for (const row of rows) {
    const rowContext: Record<string, { id: string; isConfidential: boolean }

    for (const stepConfig of config.steps) {
      if (stepConfig.target === "parent_object") {
        const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
        const cacheKey = `${stepConfig.contextKey}_${label}`;

        let parentInfo = contextCache[cacheKey];

        if (!parentInfo) {
          const [bestaand] = await tx
            .select()
            .from(schema.objects)
            .where(eq(schema.objects.label, label));

          if (bestaand) {
            parentInfo = { id: bestaand.id, isConfidential: bestaand.isConfi
            uniekeDozenGevonden++;
          } else {
            const newParentId = uuidv7();

            // 1. Maak Doos aan
            await tx.insert(schema.objects).values({
              id: newParentId,
              label: label,
              isConfidential: importIsConfidential,
              validFrom: nu,
              updatedAt: nu,
            });

            // 2. Leg ingaande relatie "is objecttype"
            // Source = ObjectType ID (bijv. Insectendoos ID)
            // Target = Nieuwe Doos ID
            if (stepConfig.objectTypeId) {
              await tx.insert(schema.relationValues).values({
                id: uuidv7(),
                relationId: IS_OBJECTTYPE_RELATION_ID,
                sourceId: stepConfig.objectTypeId, // Source: "Object type:
                targetId: newParentId, // Target: De nieuwe doo
                isConfidential: importIsConfidential,
```

```
        validFrom: nu,
        updatedAt: nu,
    });
}

// 3. Parameters (Toelichting)
if (stepConfig.parameters) {
    for (const param of stepConfig.parameters) {
        const paramVal = row[param.column];
        if (paramVal) {
            await tx.insert(schema.parameterValues).values({
                id: uuidv7(),
                parameterId: param.parameterId,
                targetId: newParentId,
                targetType: "object",
                value: paramVal,
                isConfidential: importIsConfidential,
                validFrom: nu,
                updatedAt: nu,
            });
        }
    }
}

parentInfo = { id: newParentId, isConfidential: importIsConfiden
uniekeDozenAangemaakt++;
}

contextCache[cacheKey] = parentInfo;
}

if (stepConfig.contextKey) {
    rowContext[stepConfig.contextKey] = parentInfo;
}
} else if (stepConfig.target === "child_object") {
    const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
    const childId = uuidv7();
    const childIsConfidential = importIsConfidential;

    // 1. Maak Child Object aan
    await tx.insert(schema.objects).values({
        id: childId,
        label: label,
        isConfidential: childIsConfidential,
        validFrom: nu,
        updatedAt: nu,
    });

    // 2. Leg ingaande relatie "is objecttype" voor Child Object
    if (stepConfig.objectTypeId) {
```

```
    await tx.insert(schema.relationValues).values({
      id: uuidv7(),
      relationId: IS_OBJECTTYPE_RELATION_ID,
      sourceId: stepConfig.objectTypeId, // Source: "Object type: Spec
      targetId: childId,                // Target: Het nieuwe child o
      isConfidential: childIsConfidential,
      validFrom: nu,
      updatedAt: nu,
    });
  });
}

// 3. Hiërarchische relatie (zit in doos)
if (stepConfig.relations) {
  for (const rel of stepConfig.relations) {
    const parentInfo = rowContext[rel.relatedContextKey] || contextC
    if (parentInfo) {
      const sourceId = rel.direction === "outgoing" ? parentInfo.id
      const targetId = rel.direction === "outgoing" ? childId : pare
      const relationIsConfidential = parentInfo.isConfidential || ch

      await tx.insert(schema.relationValues).values({
        id: uuidv7(),
        relationId: rel.relationTypeId,
        sourceId: sourceId,
        targetId: targetId,
        isConfidential: relationIsConfidential,
        validFrom: nu,
        updatedAt: nu,
      });
    }
  }
}

// 4. Parameters (Aantal)
if (stepConfig.parameters) {
  for (const param of stepConfig.parameters) {
    const paramVal = row[param.column];
    if (paramVal) {
      await tx.insert(schema.parameterValues).values({
        id: uuidv7(),
        parameterId: param.parameterId,
        targetId: childId,
        targetType: "object",
        value: String(paramVal),
        isConfidential: childIsConfidential,
        validFrom: nu,
        updatedAt: nu,
      });
    }
  }
}
```

```
        }

        specimengroepenAangemaakt++;
    }
}
});

revalidatePath("/");
return {
    success: true,
    stats: {
        uniekeDozenAangemaakt,
        uniekeDozenGevonden,
        specimengroepenAangemaakt,
        totaalVerwerkteRegels: rows.length,
        isConfidential: importIsConfidential,
    },
};
} catch (err: any) {
    console.error("Import execution failed:", err);
    return { success: false, error: err.message || "Fout tijdens import." };
}
}
```

Hiermee worden de ingaande **"is objecttype"** relaties exact volgens jouw domeinmodel aan de database toegevoegd!

User prompt: header moet ook nog aangepast denk ik?

Response: Ja, klopt! In de CSV-header moeten we de `objectTypeId` voor de specimengroep nog invullen met het juiste ID voor dat specifieke type (bijv. ID voor "Object type: Specimengroep").

Hier is de bijgewerkte **Metadata Header** waarin stap 1 naar het Insectendoos type wijst (019fcd20-b442-755f-af50-9cdf9716990d) en alle overige relaties en parameters van de juiste ID's zijn voorzien:

```
# {
#   "profile": "Insectendoos Specimen Import",
#   "version": "1.0",
#   "delimiter": ";",
#   "ignoreEmptyRows": true,
#   "isConfidential": false,
#   "steps": [
#     {
#       "step": 1,
#       "target": "parent_object",
#       "contextKey": "DoosId",
```

```
#       "labelTemplate": "Insectendoos: {0}",
#       "objectTypeId": "019fcd20-b442-755f-af50-9cdf9716990d",
#       "parameters": [
#         { "parameterId": "019fc74c-cf8c-74ff-a3b6-b6d21c651a19", "column": 1 }
#       ]
#     },
#     {
#       "step": 2,
#       "target": "child_object",
#       "labelTemplate": "Specimen groep: {2} {3}",
#       "objectTypeId": "019fcd20-b56f-76ae-9bf0-f98a0354b7ca",
#       "relations": [
#         {
#           "relationTypeId": "019fad01-ca30-769d-8c9c-6fc70fa9db0a",
#           "relatedContextKey": "DoosId",
#           "direction": "outgoing"
#         }
#       ],
#       "parameters": [
#         { "parameterId": "019fad6f-c149-7795-b3e5-751c8b2b7949", "column": 4 }
#       ]
#     }
#   ]
# }
"Insectendoos_label";"Insectendoos_toelichting";"Geslacht";"Soort";"Aantal"
"15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
"15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
"15-01: 17.10.5 L "; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinid
;;;
"15-02: 17.10.6L"; "Natuurmuseum, Enschede - Coleoptera Nederland - Staphylinidae
```

- **Stap 1:** Koppel de nieuw aangemaakte doos via relatietype "is objecttype" (019fcdd3-721a-7512-b755-cddd67f43eb6) aan Source Object Type Insectendoos (019fcd20-b442-755f-af50-9cdf9716990d).
- **Stap 2:** Koppel de nieuwe specimengroep via relatietype "is objecttype" aan Source Object Type Specimengroep (019fcd20-b56f-76ae-9bf0-f98a0354b7ca).

(Mocht het ID voor Object Type "Specimengroep" in jouw database anders zijn dan 019fcd20-b56f-76ae-9bf0-f98a0354b7ca, dan hoeft je alleen die specifieke UUID in de JSON aan te passen).

User prompt: ik zie bij testen nu dat omkeren van de relatie (source en target verwisselen) niet werkt.

Response: Het probleem zit in `bewerkRelatie` in `app/actions.ts`. Er zijn twee kolomnamen die afwijken van het Drizzle schema:

1. **Kolomnamen:** De code gebruikt `currentRel.sourceObjectId` en `targetObjectId`, terwijl de velden in het schema `sourceId` en `targetId` heten.
2. **Volgordeherberekening:** Bij het omdraaien van de richting verandert de `sourceId`. Daardoor moet er voor het nieuwe bron-object ook een nieuwe `volgorde` bepaald worden, anders valt de relatie buiten de verwachte sorteervolgorde.

Oplossing voor `app/actions.ts`

Vervang de functie `bewerkRelatie` in `app/actions.ts` door deze versie:

```
// app/actions.ts

export async function bewerkRelatie({
  relationValueId,
  nieuwRelationId,
  wisselRichting,
  validFrom,
  validTo,
}): {
  relationValueId: string;
  nieuwRelationId?: string;
  wisselRichting?: boolean;
  validFrom?: string | null;
  validTo?: string | null;
}) {
  try {
    const client = activeDb || db;
    if (!client) return { success: false, error: "Geen actieve database." };

    // 1. Ophalen van de huidige relatie
    const currentRel = await client
      .select()
      .from(schema.relationValues)
      .where(eq(schema.relationValues.id, relationValueId))
      .get();

    if (!currentRel) {
      return { success: false, error: "Relatie niet gevonden." };
    }

    // 2. Bepaal de nieuwe waarden
    const targetRelationId = nieuwRelationId || currentRel.relationId;

    // Gebruik de correcte schemanamen: sourceId en targetId
    let newSourceId = currentRel.sourceId;
    let newTargetId = currentRel.targetId;
    let newVolgorde = currentRel.volgorde;

    // Als de richting omgewisseld wordt
```

```
if (wisselRichting) {
  newSourceId = currentRel.targetId;
  newTargetId = currentRel.sourceId;

  // Bepaal de hoogste volgorde bij het nieuwe bron-object
  const bestaandeRelaties = await client
    .select({ maxVolgorde: max(schema.relationValues.volgorde) })
    .from(schema.relationValues)
    .where(eq(schema.relationValues.sourceId, newSourceId));

  const hoogsteVolgorde = bestaandeRelaties[0]?.maxVolgorde ?? 0;
  newVolgorde = hoogsteVolgorde + 1;
}

// 3. Database Update uitvoeren
await client
  .update(schema.relationValues)
  .set({
    relationId: targetRelationId,
    sourceId: newSourceId,
    targetId: newTargetId,
    volgorde: newVolgorde,
    validFrom: validFrom !== undefined ? validFrom : currentRel.validFrom,
    validTo: validTo !== undefined ? validTo : currentRel.validTo,
    updatedAt: new Date().toISOString(),
  })
  .where(eq(schema.relationValues.id, relationValueId));

// Herbereken de volgorde van het oude bron-object bij richting-wissel
if (wisselRichting) {
  await hernummerVolgordeVoorSource(client, currentRel.sourceId);
}

revalidatePath("/");
return { success: true };
} catch (error: any) {
  console.error("Fout bij bewerken relatie:", error);
  return { success: false, error: error.message || "Fout bij opslaan." };
}
}
```

User prompt: Ik ben even in de details in de database gedoken. De basisroutine bij nieuwe relatiewaarde is dat volgnummer automatisch laatste + 1 wordt (uitgaande). Nu wordt in de CSV import volgnummer = 0 gegeven

Response: Dat verklaart waarom de volgorde bij de geïmporteerde relaties niet klopt.

Als er voor een `sourceId` al bestaande relaties zijn (of meerdere relaties in dezelfde import-batch

voor hetzelfde bron-object worden aangemaakt), moet het volgnummer net als bij `voegRelatieToe` dynamisch berekend worden op basis van de al aanwezige uitgaande relaties.

Hier is hoe je de transactie in `app/import/actions.ts` aanpast zodat de volgorde netjes wordt opgehoogd.

Aanpassing in `app/import/actions.ts`

Voeg een hulpfunctie toe om het volgende beschikbare volgnummer voor een specifieke `sourceId` bij te houden en direct op te hogen in de transactie:

```
// app/import/actions.ts
"use server";

import { activeDb as db } from "@src/lib/db";
import * as schema from "@src/db/schema";
import { eq, max } from "drizzle-orm";
import { ImportConfigHeader } from "@src/lib/csvParser";
import { revalidatePath } from "next/cache";
import { v7 as uuidv7 } from "uuid";

const IS_OBJECTTYPE_RELATION_ID = "019fcdd3-721a-7512-b755-cddd67f43eb6";

export async function voerCsvImportUit({
  config,
  rows,
}: {
  config: ImportConfigHeader;
  rows: string[][];
}) {
  try {
    let uniekeDozenAangemaakt = 0;
    let uniekeDozenGevonden = 0;
    let specimengroepenAangemaakt = 0;

    const importIsConfidential = Boolean(config.isConfidential);
    const contextCache: Record<string, { id: string; isConfidential: boolean }>

    // Cache om volgnummers per sourceId in te bewaren gedurende de transactie
    const volgordeTracker: Record<string, number> = {};

    await db.transaction(async (tx: any) => {
      const nu = new Date().toISOString();

      // Hulpfunctie om volgend volgnummer op te halen en op te hogen
      const getVolgendVolgnummer = async (sourceId: string): Promise<number> =>
        if (volgordeTracker[sourceId] === undefined) {
          const [bestaandeRelatie] = await tx
            .select({ maxVolgorde: max(schema.relationValues.volgorde) })
            .from(schema.relationValues)
```



```

        .where(eq(schema.relationValues.sourceId, sourceId));

    volgordeTracker[sourceId] = bestaandeRelatie?.maxVolgorde ?? 0;
}

volgordeTracker[sourceId] += 1;
return volgordeTracker[sourceId];
};

for (const row of rows) {
    const rowContext: Record<string, { id: string; isConfidential: boolean }

    for (const stepConfig of config.steps) {
        if (stepConfig.target === "parent_object") {
            const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
            const cacheKey = `${stepConfig.contextKey}_${label}`;

            let parentInfo = contextCache[cacheKey];

            if (!parentInfo) {
                const [bestaand] = await tx
                    .select()
                    .from(schema.objects)
                    .where(eq(schema.objects.label, label));

                if (bestaand) {
                    parentInfo = { id: bestaand.id, isConfidential: bestaand.isConfi
                    uniekeDozenGevonden++;
                } else {
                    const newParentId = uuidv7();

                    // 1. Parent Object
                    await tx.insert(schema.objects).values({
                        id: newParentId,
                        label: label,
                        isConfidential: importIsConfidential,
                        validFrom: nu,
                        updatedAt: nu,
                    });

                    // 2. Relatie "is objecttype" (met automatisch volnummer)
                    if (stepConfig.objectTypeId) {
                        const volgorde = await getVolgendVolnummer(stepConfig.objectT

                        await tx.insert(schema.relationValues).values({
                            id: uuidv7(),
                            relationId: IS_OBJECTTYPE_RELATION_ID,
                            sourceId: stepConfig.objectTypeId,
                            targetId: newParentId,
                            volgorde: volgorde,

```

```

        isConfidential: importIsConfidential,
        validFrom: nu,
        updatedAt: nu,
    });
}

// 3. Parameters
if (stepConfig.parameters) {
    for (const param of stepConfig.parameters) {
        const paramVal = row[param.column];
        if (paramVal) {
            await tx.insert(schema.parameterValues).values({
                id: uuidv7(),
                parameterId: param.parameterId,
                targetId: newParentId,
                targetType: "object",
                value: paramVal,
                isConfidential: importIsConfidential,
                validFrom: nu,
                updatedAt: nu,
            });
        }
    }
}

parentInfo = { id: newParentId, isConfidential: importIsConfiden
uniekeDozenAangemaakt++;
}

contextCache[cacheKey] = parentInfo;
}

if (stepConfig.contextKey) {
    rowContext[stepConfig.contextKey] = parentInfo;
}
} else if (stepConfig.target === "child_object") {
    const label = stepConfig.labelTemplate.replace(/\{(\d+)\}/g, (_, ind
    const childId = uuidv7();
    const childIsConfidential = importIsConfidential;

    // 1. Child Object
    await tx.insert(schema.objects).values({
        id: childId,
        label: label,
        isConfidential: childIsConfidential,
        validFrom: nu,
        updatedAt: nu,
    });

    // 2. Relatie "is objecttype" (met automatisch volgnummer)

```

```
if (stepConfig.objectTypeId) {
  const volgorde = await getVolgendVolgnummer(stepConfig.objectTypeI

  await tx.insert(schema.relationValues).values({
    id: uuidv7(),
    relationId: IS_OBJECTTYPE_RELATION_ID,
    sourceId: stepConfig.objectTypeId,
    targetId: childId,
    volgorde: volgorde,
    isConfidential: childIsConfidential,
    validFrom: nu,
    updatedAt: nu,
  });
}

// 3. Hiërarchische relatie (zit in doos - met automatisch volgnummer
if (stepConfig.relations) {
  for (const rel of stepConfig.relations) {
    const parentInfo = rowContext[rel.relatedContextKey] || contextC
    if (parentInfo) {
      const sourceId = rel.direction === "outgoing" ? parentInfo.id
      const targetId = rel.direction === "outgoing" ? childId : pare
      const relationIsConfidential = parentInfo.isConfidential || ch

      const volgorde = await getVolgendVolgnummer(sourceId);

      await tx.insert(schema.relationValues).values({
        id: uuidv7(),
        relationId: rel.relationTypeId,
        sourceId: sourceId,
        targetId: targetId,
        volgorde: volgorde,
        isConfidential: relationIsConfidential,
        validFrom: nu,
        updatedAt: nu,
      });
    }
  }
}

// 4. Parameters
if (stepConfig.parameters) {
  for (const param of stepConfig.parameters) {
    const paramVal = row[param.column];
    if (paramVal) {
      await tx.insert(schema.parameterValues).values({
        id: uuidv7(),
        parameterId: param.parameterId,
        targetId: childId,
        targetType: "object",
      });
    }
  }
}
```

```

        value: String(paramVal),
        isConfidential: childIsConfidential,
        validFrom: nu,
        updatedAt: nu,
    });
    }
}
}

specimengroepenAangemaakt++;
}
}
});

revalidatePath("/");
return {
    success: true,
    stats: {
        uniekeDozenAangemaakt,
        uniekeDozenGevonden,
        specimengroepenAangemaakt,
        totaalVerwerkteRegels: rows.length,
        isConfidential: importIsConfidential,
    },
};
} catch (err: any) {
    console.error("Import execution failed:", err);
    return { success: false, error: err.message || "Fout tijdens import." };
}
}

```

Wat dit verandert:

- Bij elke relatie-insert wordt via `getVolgendVolgnummer(sourceId)` gecontroleerd wat het op dat moment hoogste volgnummer is in de database voor die `sourceId`.
- De interne `volgordeTracker` update de teller direct in het geheugen tijdens de import, zodat meerdere childs in dezelfde import voor dezelfde doos netjes ge geordend worden op 1, 2, 3, etc.

User prompt: Voor mij is nu de grote vraag of en hoe ik bij een volgende CSV zelf de metadata header kan schrijven.

Response: Het schrijven van je eigen metadata-header in de CSV is erg overzichtelijk zodra je de opbouw van de JSON begrijpt. De header staat bovenaan de CSV, begint op elke regel met een # en heeft een vaste structuur.

Basic JSON Structuur

```
# {
#   "profile": "Naam van je importprofiel",
#   "version": "1.0",
#   "delimiter": ";",
#   "ignoreEmptyRows": true,
#   "isConfidential": false,
#   "steps": [ ... ]
# }
```

- **delimiter**: Het scheidingsteken in je CSV (meestal ";" of ",").
- **isConfidential**: Zet op `true` als alle geïmporteerde objecten/relaties/parameters als vertrouwelijk gemarkeerd moeten worden.
- **steps**: Een array van stappen die per CSV-regel in volgorde worden uitgevoerd.

De twee typen stappen (steps)

Een stap representeert een te maken of te koppelen object (bijv. een Hoofd-object of een Sub-object).

1. Het Hoofd-object (parent_object)

Maakt een nieuw hoofdobject aan of zoekt een bestaande op basis van het geopende label (ontdubbeling op `labelTemplate`).

```
{
  "step": 1,
  "target": "parent_object",
  "contextKey": "DoosId",
  "labelTemplate": "Insectendoos: {0}",
  "objectTypeId": "<UUID_VAN_OBJECTTYPE_INSECTENDOOS>",
  "parameters": [
    { "parameterId": "<UUID_PARAMETER_TOELICHTING>", "column": 1 }
  ]
}
```

- **contextKey**: Een unieke sleutel (bijv. "DoosId"). Hiermee kunnen vervolgstappen refereren naar dit specifiek aangemaakte/gevonden parent-object.
- **labelTemplate**: Formaat voor de naam van het object. `{0}` leest waarde uit CSV-kolom 0 (eerste kolom), `{1}` uit kolom 1, etc.
- **objectTypeId**: Het ID van het Type Object in je database (bijv. ID van "Object type: Insectendoos").
- **parameters**: Lijst van toe te voegen parameters:

- **parameterId**: Het ID van de parameter in de database.
- **column**: Het 0-indexed kolomnummer in de CSV waar de waarde staat (0 = 1e kolom, 1 = 2e kolom, etc.).

2. Het Kind-object (**child_object**)

Maakt voor **elke** regel een nieuw child-object aan en koppelt dit aan eerdere stappen via relaties.

```
{
  "step": 2,
  "target": "child_object",
  "labelTemplate": "Specimen groep: {2} {3}",
  "objectTypeId": "<UUID_VAN_OBJECTTYPE_SPECIMENGROEP>",
  "relations": [
    {
      "relationTypeId": "<UUID_RELATIETYPE_LIGT_IN>",
      "relatedContextKey": "DoosId",
      "direction": "outgoing"
    }
  ],
  "parameters": [
    { "parameterId": "<UUID_PARAMETER_AANTAL>", "column": 4 }
  ]
}
```

- **relations**: Legt een relatie tussen het nieuwe child-object en een eerdere stap:
 - **relationTypeId**: Het ID van de relatie-definitie in de database (bijv. "ligt/zit in of op").
 - **relatedContextKey**: Koppelt met de **contextKey** uit een eerdere stap (bijv. "DoosId").
 - **direction**:
 - "outgoing": Parent is Bron → Child is Target.
 - "incoming": Child is Bron → Parent is Target.

Stappenplan voor een nieuwe CSV

1. Haal de benodigde UUID's op uit de database:

- ID van het relatietype tussen objecten (bijv. "ligt in").
- ID's van de Object Types (bijv. "Locatie", "Apparaat").
- ID's van de Parameters die je wilt vullen (bijv. "Serienummer", "Status").

2. Tel de kolommen van je CSV vanaf index 0:

- Kolom 1 = {0} / column: 0
- Kolom 2 = {1} / column: 1
- Kolom 3 = {2} / column: 2

3. Bouw de JSON op:

- Bepaal wat het bovenliggende object is (`parent_object`) en wat de onderliggende objecten zijn (`child_object`).
- Vul de `labelTemplates` in met de gewenste kolom-placeholders {X}.
- Voeg de JSON toe als opmerkingen (# ...) helemaal aan het begin van het CSV-bestand.

User prompt: Als ik de code bekijk zie ik `IS_OBJECTTYPE_RELATION_ID = "019fcdd3-721a-7512-b755-cddd67f43eb6"` staan in `actions.ts`. Dit betekent toch dat we dit deel hard coderen. Ik zie nergens in de `MetadataHeader` iet staan over het aanmaken van nieuwe objecten op basis van CSV, er worden alleen relaties gelegd en parameters gekoppeld; klopt dat?

Response: Dat klopt precies. Je observeert twee heel belangrijke zaken over het huidige ontwerp:

1. `IS_OBJECTTYPE_RELATION_ID` is nu hardgecodeerd Dat is inderdaad hardgecodeerd in de Server Action. Het idee hierachter was dat de relatie "is van objecttype X" een vaste systeemrelatie is binnen de applicatie. Maar dit betekent wel dat als die UUID ooit verandert of per omgeving anders is, de code aangepast moet worden.

2. CSV Import maakt wél nieuwe objecten aan De import maakt wel degelijk nieuwe objecten aan in de tabel `schema.objects`, maar de metadata-header is zo ontworpen dat een "stap" (step) direct de definitie vormt van een object.

Elke stap in de `steps`-array definieert welk object er moet ontstaan of gezocht moet worden:

- **In de code (`actions.ts`):** Wanneer de lus door een stap loopt, voert hij eerst dit uit:

```
await tx.insert(schema.objects).values({
  id: childId,
  label: label, ...
});
```

Het **object zelf** krijgt zijn naam/label via de `labelTemplate` (bijvoorbeeld `labelTemplate: "Specimen groep: {2} {3}"`).

- **In de Metadata Header:** Je geeft in de JSON niet expliciet een commando "createObject", maar het specificeren van een stap met `target: "parent_object"` of `target: "child_object"` is de instructie om dat object aan te maken.

De overige velden in die stap vertellen de importeur vervolgens welke extra zaken direct aan het nieuw aangemaakte object gekoppeld moeten worden:

1. **objectTypeId**: Welk type object is het? (maakt de "is objecttype" relatie aan).
2. **relations**: Welke relatie heeft het met andere stappen/objecten?
3. **parameters**: Welke eigenschappen/meetwaarden uit de CSV-kolommen moeten op dit object gezet worden?

Flexibeler maken?

Als je wilt dat het relatietype ID voor "is objecttype" niet hardgecodeerd staat in TypeScript, zou je dat veld ook naar de metadata-header kunnen verplaatsen (bijvoorbeeld "objectTypeRelationId": "019fcdd3..." in het root-JSON-object van de header).

User prompt: De allereerste stap was de check of de insectendoos al bestond of niet, dat gaat in wezen toch vooraf aan stap 1? Waar komt anders DoosId vandaan?

Response: Dat voelt inderdaad tegen-intuïtief, maar in de opzet van dit importer-systeem is **Stap 1 (step: 1) zélf de check én de creatie/toewijzing van DoosId**.

Stap 1 heeft de rol van **parent_object**. Het stappenplan verwerkt Stap 1 als volgt:

1. **Label genereren**: Stap 1 leest de CSV-regel en bouwt het label op via de `labelTemplate` (bijvoorbeeld "Insectendoos: 15-01: 17.10.5 L").
2. **Zoeken (De check vooraf)**: De code voert eerst een `tx.select()` uit op de database om te kijken of er al een object bestaat met exact dat label.
3. **Vertakking**:
 - **Bestaat de doos al?** → De code pakt de bestaande `id` van die doos uit de database en slaat deze in het geheugen op onder de sleutel `contextKey: "DoosId"`. Hij maakt **geen** nieuw object of type-relatie aan.
 - **Bestaat de doos nog niet?** → De code genereert een nieuwe UUID, maakt het nieuw `schema.objects` record aan in de database, legt de `objectTypeId` relatie én slaat die nieuwe UUID op in het geheugen onder `contextKey: "DoosId"`.

Hoe 'DoosId' beschikbaar komt voor Stap 2

Aan het einde van Stap 1 is `DoosId` dus **gegarandeerd gevuld** in het geheugen van de importeur (met een bestaand ID óf een gloednieuw ID).

Wanneer de importeur vervolgens doorgaat naar **Stap 2 (child_object)**, leest hij de relatie-configuratie:

```
"relations": [  
  {  
    "relationTypeId": "019fad01-...",
```



```
    "relatedContextKey": "DoosId",  
    "direction": "outgoing"  
  }  
]
```

Stap 2 pakt de opgeslagen ID achter `relatedContextKey: "DoosId"` en gebruikt die als `sourceId` om de `relationValues` record aan te maken richting het nieuwe child-object.

Kort samengevat:

Er is geen aparte stap vóór Stap 1 nodig; het **zoeksysteem (ontdubbelen)** zit **direct ingebouwd in het type `parent_object`** van Stap 1.
